

1. (a) Consider the recurrence relation for Strassen's matrix multiplication algorithm. (2)

$$T(n) = 7T(n/2) + \Theta(n^2)$$

Explain which operations contribute to the cost component $\Theta(n^2)$.

Gemini 3.1 Pro Extended

Here is the explanation for the cost component asked in the image.

Breakdown of the Recurrence Relation

The recurrence relation for Strassen's algorithm is: $T(n) = 7T(n/2) + \Theta(n^2)$

- $7T(n/2)$: Represents the cost of the 7 recursive matrix multiplications, each of size $(n/2) \times (n/2)$.
- $\Theta(n^2)$: Represents the cost of the extra work done outside the recursive calls (the "divide" and "combine" steps).

Operations Contributing to $\Theta(n^2)$

The $\Theta(n^2)$ cost component comes primarily from the **matrix additions and subtractions** required to set up the recursive calls and to construct the final matrix from the results.

Specifically, this component accounts for:

1. **Pre-multiplication (Additions/Subtractions)**: Before making the 7 recursive multiplications, the algorithm creates 10 intermediate matrices by adding or subtracting specific quadrants of the original matrices. Each quadrant is of size $(n/2) \times (n/2)$. Adding or subtracting matrices of this size requires $\Theta((n/2)^2) = \Theta(n^2)$ operations.
2. **Post-multiplication (Combining Results)**: After the 7 recursive multiplications are complete, the algorithm performs 8 more matrix additions and subtractions on those results to calculate the four quadrants of the final $n \times n$ product matrix. This also takes $\Theta(n^2)$ time.
3. **Partitioning**: Logically dividing the original $n \times n$ matrices into four submatrices takes $\Theta(1)$ time if done via index manipulation, but if the algorithm implementation allocates new arrays and copies the data, this step also contributes to the $\Theta(n^2)$ cost.

In total, Strassen's algorithm performs a constant number (18) of matrix additions and subtractions on $(n/2) \times (n/2)$ matrices. Because $18 \times \Theta(n^2/4)$ simplifies asymptotically, the entire non-recursive workload is bounded by $\Theta(n^2)$.

(c) Demonstrate through an array of length 5 that the selection sort algorithm is not stable. (3)

(d) Give an example of a 4-node graph that does not have a topological ordering. (3)

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image.

(c) Demonstrate through an array of length 5 that the selection sort algorithm is not stable.

A sorting algorithm is considered **stable** if it preserves the relative order of equal elements. If two elements have the same value, the one that appeared first in the unsorted array should appear first in the sorted array. Selection sort fails this because swapping elements over long distances can displace an equal element.

Let's demonstrate this with an array of length 5. We will use `4a` and `4b` to track two elements with the same numerical value (4) to see if their relative order changes.

Initial Array: `[4a, 4b, 3, 2, 1]` (Note: `4a` comes before `4b`)

- **Pass 1:** The algorithm finds the minimum element in the array, which is `1` (at index 4). It swaps `1` with the element at the first position `4a` (at index 0).
 - **Array becomes:** `[1, 4b, 3, 2, 4a]`
 - Notice that `4a` has now been pushed behind `4b` .
- **Pass 2:** The algorithm finds the minimum element in the remaining unsorted portion `[4b, 3, 2, 4a]` , which is `2` . It swaps `2` with `4b` .
 - **Array becomes:** `[1, 2, 3, 4b, 4a]`
- **Pass 3:** The minimum in `[3, 4b, 4a]` is `3` . It is already in the correct spot, so it swaps with itself.
 - **Array becomes:** `[1, 2, 3, 4b, 4a]`
- **Pass 4:** The minimum in `[4b, 4a]` is `4b` . It swaps with itself.
 - **Final Sorted Array:** `[1, 2, 3, 4b, 4a]`

Conclusion: In the original array, `4a` appeared before `4b` . In the final sorted array, `4b` appears before `4a` . Because the relative order of equal elements was not maintained, this proves that selection sort is **not stable**.

(d) Give an example of a 4-node graph that does not have a topological ordering.

For a directed graph to have a valid topological ordering, it must be a **Directed Acyclic Graph (DAG)**—meaning it cannot contain any directed cycles. If a graph has a cycle, it creates an impossible dependency loop where a node ultimately depends on itself, preventing a linear ordering.

To answer the question, we simply need to construct a 4-node directed graph that includes at least one cycle.

Example Graph:

- **Vertices (Nodes):** $V = \{A, B, C, D\}$

- **Directed Edges:** $E = \{(A \rightarrow B), (B \rightarrow C), (C \rightarrow A), (C \rightarrow D)\}$

Why it lacks a topological ordering: This graph contains the directed cycle $A \rightarrow B \rightarrow C \rightarrow A$. If you tried to write a topological ordering, Node A must come before Node B, Node B must come before Node C, and Node C must come before Node A. This is a logical contradiction, so no topological ordering can exist.

You Asked 

(h) Show that the greedy method based on profit-to-weight ratio does not always give an optimal solution for the 0-1 knapsack problem? (4)

(i) Establish the correctness of the insertion sort algorithm using an appropriate loop invariant. (5)

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image.

(h) Show that the greedy method based on profit-to-weight ratio does not always give an optimal solution for the 0-1 knapsack problem?

To prove this, we can use a counterexample. The 0-1 knapsack problem restricts us to either taking an entire item or leaving it completely; we cannot take fractions of an item.

The greedy approach based on the profit-to-weight ratio dictates that we should always sort the items in descending order of their ratio ($\frac{\text{Profit}}{\text{Weight}}$) and pick the items with the highest ratio first until the knapsack is full.

Counterexample Setup:

- **Knapsack Capacity (W):** 50 kg
- **Item 1:** Profit = 60, $Weight = 10kg \rightarrow$ Ratio = 6.0
- **Item 2:** Profit = 100, $Weight = 20kg \rightarrow$ Ratio = 5.0
- **Item 3:** Profit = 120, $Weight = 30kg \rightarrow$ Ratio = 4.0

1. The Greedy Approach (Profit-to-Weight Ratio):

- Sort by ratio: Item 1 (6.0), then Item 2 (5.0), then Item 3 (4.0).
- **Pick Item 1:** Weight becomes 10, Profit = 60. Remaining capacity = 40.
- **Pick Item 2:** Weight becomes $10 + 20 = 30$, Profit = $60 + 100 = 160$. Remaining capacity = 20.
- **Attempt Item 3:** Weight is 30, but we only have 20 capacity left. Because this is the 0-1 knapsack problem, we cannot take a fraction of Item 3. We must skip it.
- **Total Greedy Profit = \$160**

2. The Optimal Approach:

- If we ignore the highest ratio (Item 1) and instead choose Item 2 and Item 3:
- Total Weight = 20 kg + 30 kg = 50 kg (Exactly fills the capacity)
- **Total Optimal Profit = \$100 + 120 = 220**

Conclusion: Since the optimal profit (220) *is strictly greater than the profit found by the greedy approach* (160), this demonstrates that sorting by the profit-to-weight ratio does not guarantee an optimal solution for the 0-1 knapsack problem.

(i) Establish the correctness of the insertion sort algorithm using an appropriate loop invariant.

To establish correctness, we must define a **loop invariant** (a condition that is true before and after every iteration of the loop) and prove three properties: **Initialization**, **Maintenance**, and **Termination**.

Consider the standard Insertion Sort algorithm indexing from 1 to n :

```
text

for j = 2 to A.length:
    key = A[j]
    i = j - 1
    while i > 0 and A[i] > key:
        A[i+1] = A[i]
        i = i - 1
    A[i+1] = key
```

The Loop Invariant: At the start of each iteration of the outer `for` loop, the subarray $A[1 \dots j - 1]$ consists of the elements originally in $A[1 \dots j - 1]$, but they are now in sorted order.

1. Initialization (Base Case): We must show the invariant holds before the very first loop iteration begins, when $j = 2$.

- The subarray is $A[1 \dots j - 1]$, which is just $A[1]$.
- A single element is trivially sorted, and it is the original element from index 1.
- Therefore, the loop invariant holds prior to the first iteration.

2. Maintenance (Inductive Step): We must show that if the invariant is true before an iteration, it remains true before the next iteration.

- Assume the subarray $A[1 \dots j - 1]$ is sorted.
- The body of the `for` loop takes $A[j]$ (stored in `key`) and compares it with the elements in $A[1 \dots j - 1]$ from right to left.
- The `while` loop shifts all elements that are greater than `key` one position to the right to make room, and then inserts `key` into its correct sorted position.
- After this process, the subarray $A[1 \dots j]$ contains the exact same elements originally in $A[1 \dots j]$, but they are now in sorted order.
- When j increments for the next iteration, the invariant is preserved.

3. Termination: We must examine what happens when the loop finally terminates and show that this proves the algorithm is correct.

- The outer `for` loop terminates when $j > A.length$. Assuming $A.length = n$, the loop terminates when $j = n + 1$.
- Substituting $n + 1$ for j in the loop invariant, the statement becomes: "The subarray $A[1 \dots n]$ consists of the elements originally in $A[1 \dots n]$, but in sorted order."

- Since the subarray $A[1 \dots n]$ is the entire array, we have mathematically proven that the entire array is now sorted, establishing the correctness of the insertion sort algorithm.

You Asked 

(b) Given an undirected graph $G = (V, E)$, a source node S and a target node T , can we use the DFS algorithm to find the shortest path from S to T ? Justify your answer. (5)

Gemini 3.1 Pro Extended

Based on the question provided in the image, here is the answer and justification.

Answer

No, the standard Depth-First Search (DFS) algorithm cannot be reliably used to find the shortest path from a source node S to a target node T in a general unweighted graph.

Justification

1. Exploration Strategy (Deep vs. Broad) DFS explores a graph by plunging as deep as possible along a single branch until it hits a dead end, and only then does it backtrack. It does not evaluate nodes based on their distance from the source. Because of this "deep-dive" approach, the first path DFS finds to the target node T is simply the path dictated by the order in which neighbors are processed, which is highly unlikely to be the most direct route.

2. The "Visited" Constraint In standard DFS, once a node is visited, it is marked as such and is not revisited. If DFS takes a long, winding detour and reaches node T , it will mark T as visited. Even if a direct edge exists connecting S to T , DFS will ignore it later because T has already been processed, leaving you stuck with the suboptimal path.

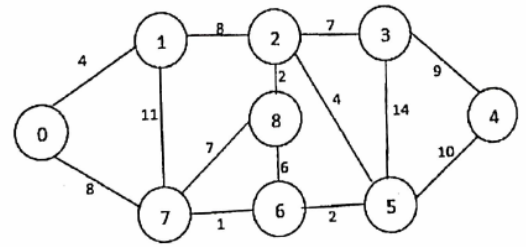
3. A Simple Counterexample Imagine an undirected graph with nodes $V = \{S, A, B, T\}$ and edges $E = \{(S, A), (A, B), (B, T), (S, T)\}$.

- The actual shortest path from S to T is the direct edge $S \rightarrow T$ (length of 1).
- If you run DFS from S , and the algorithm happens to evaluate neighbor A before neighbor T , it will follow the path $S \rightarrow A \rightarrow B \rightarrow T$ (length of 3). It will declare this as the path to T , completely missing the direct edge.

(Note: The correct algorithm to find the shortest path in an unweighted graph is **Breadth-First Search (BFS)**, because it explores the graph level-by-level, ensuring that the first time it encounters the target node, it has taken the shortest possible route).

You Asked 

- (c) For the graph given below, find a minimum spanning tree using the Prim's algorithm considering node 0 as the start node. Also, state the time complexity of the same. (5)



Gemini 3.1 Pro Extended

Based on the graph provided in the image, here is the step-by-step solution for finding the Minimum Spanning Tree (MST) using Prim's Algorithm, followed by its time complexity.

Part 1: Prim's Algorithm Execution

Goal: Find a connected subgraph that includes all vertices, contains no cycles, and has the minimum possible total edge weight.

Initialization: * Start Node: 0

- Nodes currently in MST: {0}

We will systematically pick the smallest edge that connects a node inside the MST to an unvisited node outside the MST.

- Step 1:**

- Available edges leaving MST: (0,1) weight 4; (0,7) weight 8.
- Minimum edge: **(0, 1)** with weight 4.
- Nodes in MST: {0, 1}

- Step 2:**

- Available edges: (0,7) w=8; (1,2) w=8; (1,7) w=11.
- Minimum edge: There is a tie between (0,7) and (1,2). We can pick either. Let's pick **(0, 7)** with weight 8.
- Nodes in MST: {0, 1, 7}

- Step 3:**

- Available edges: (1,2) w=8; (1,7) w=11 (creates cycle, ignore); (7,6) w=1; (7,8) w=7.
- Minimum edge: **(7, 6)** with weight 1.
- Nodes in MST: {0, 1, 7, 6}

- Step 4:**

- Available edges: (1,2) w=8; (7,8) w=7; (6,5) w=2; (6,8) w=6.
- Minimum edge: **(6, 5)** with weight 2.

- Nodes in MST: {0, 1, 7, 6, 5}
- **Step 5:**
 - Available edges: (1,2) w=8; (7,8) w=7; (6,8) w=6; (5,2) w=4; (5,3) w=14; (5,4) w=10.
 - Minimum edge: **(5, 2)** with weight 4.
 - Nodes in MST: {0, 1, 7, 6, 5, 2}
- **Step 6:**
 - Available edges: (1,2) w=8 (creates cycle, ignore); (7,8) w=7; (6,8) w=6; (2,8) w=2; (2,3) w=7; (5,3) w=14; (5,4) w=10.
 - Minimum edge: **(2, 8)** with weight 2.
 - Nodes in MST: {0, 1, 7, 6, 5, 2, 8}
- **Step 7:**
 - Available edges: (7,8) and (6,8) (create cycles, ignore); (2,3) w=7; (5,3) w=14; (5,4) w=10.
 - Minimum edge: **(2, 3)** with weight 7.
 - Nodes in MST: {0, 1, 7, 6, 5, 2, 8, 3}
- **Step 8:**
 - Available edges: (5,3) (creates cycle, ignore); (5,4) w=10; (3,4) w=9.
 - Minimum edge: **(3, 4)** with weight 9.
 - Nodes in MST: {0, 1, 7, 6, 5, 2, 8, 3, 4}

All nodes have now been visited.

Final Minimum Spanning Tree Summary:

- **Edges included:** (0,1), (0,7), (7,6), (6,5), (5,2), (2,8), (2,3), (3,4)
- **Total Minimum Cost:** $4 + 8 + 1 + 2 + 4 + 2 + 7 + 9 = 37$ (Note: If the tie in Step 2 was broken by picking edge (1,2) instead of (0,7), the final weight would still be exactly 37, which is a valid alternative MST).

Part 2: Time Complexity

The time complexity of Prim's Algorithm depends entirely on the data structures used for its implementation:

1. $O(|V|^2)$: This is the complexity if the graph is represented as an Adjacency Matrix and we use standard linear searching to find the minimum weight edge at each step (where $|V|$ is the number of vertices).
2. $O(|E| \log |V|)$: This is the more optimized and common complexity, achieved when the graph is represented using an Adjacency List and a **Min-Priority Queue (Binary Heap)** is used to extract the minimum edge (where $|E|$ is the number of edges and $|V|$ is the number of vertices).

You Asked 

3. (a) Compare the worst-case time complexities of the standard and randomized versions of Quick Sort algorithm. In standard version of Quicksort, assume that the last element is always chosen as the pivot. Explain how the use of randomization impacts the worst-case time complexity. (5)

Gemini 3.1 Pro Extended

Here is the comparison and explanation of the standard and randomized versions of QuickSort.

Comparison of Worst-Case Time Complexities

Both the standard and randomized versions of QuickSort share the exact same worst-case time complexity: $O(n^2)$.

The fundamental difference lies not in the *value* of the worst-case complexity, but in the **probability of it occurring** based on the input data.

1. Standard QuickSort (Last Element Pivot)

- **Worst-Case Complexity:** $O(n^2)$
- **When it occurs:** This happens when the input array is already strictly sorted (either ascending or descending) or when all elements are identical.
- **Why it happens:** Because the pivot is always chosen as the last element, a sorted array guarantees that the pivot is always the absolute largest (or smallest) element in the current subarray. This results in the worst possible partitioning: one subproblem of size $n - 1$ and another of size 0.
- **Recurrence Relation:** $T(n) = T(n - 1) + T(0) + \Theta(n)$ Solving this recurrence yields $T(n) = \Theta(n^2)$.

2. Randomized QuickSort

- **Worst-Case Complexity:** $O(n^2)$
- **When it occurs:** This happens only if the random number generator is extraordinarily unlucky and happens to select the largest or smallest element as the pivot during *every single* recursive call.
- **Why it happens:** Just like the standard version, if the partitions are constantly extremely unbalanced (sizes $n - 1$ and 0), the recursion tree becomes a straight line of depth n , leading to quadratic time.

The Impact of Randomization

While randomization does not technically change the $O(n^2)$ worst-case bound, it drastically impacts the algorithm's practical performance by **breaking the dependency on the input data**.

1. **Protects against pathological inputs:** In standard QuickSort, an adversary can easily force the $O(n^2)$ worst-case by deliberately feeding the algorithm an already-sorted array. Randomization prevents this. Because the pivot is chosen uniformly at random, no specific input sequence can consistently force the worst-case behavior.
2. **Guarantees $O(n \log n)$ Expected Time:** By choosing a random pivot, the algorithm is highly likely to pick a pivot near the median, resulting in reasonably balanced splits (e.g., a 1-to-9 split is still balanced enough to ensure a logarithmic tree depth). Therefore, the *expected* (average) running time for *any* input array becomes $O(n \log n)$.

In short, randomization shifts the worst-case scenario from being a common flaw caused by structured data (like sorted lists) to a practically impossible mathematical anomaly caused by astronomical bad luck.

You Asked 

(c) Run the build-max-heap procedure (that internally uses the max-heapify procedure) on the following array and report the number of swap operations done: $\langle 89, 19, 50, 27, 32, 65, 2, 5, 7 \rangle$ (5)

Gemini 3.1 Pro Extended

Here is the step-by-step execution of the `build-max-heap` procedure on the given array.

Initial Setup

- **Initial Array (A):** `[89, 19, 50, 27, 32, 65, 2, 5, 7]`
- **Array length (n):** 9

To build a max heap, the procedure identifies the last non-leaf node and calls `max-heapify` on each node from right-to-left, bottom-to-top. The index of the last non-leaf node is $\lfloor n/2 \rfloor = \lfloor 9/2 \rfloor = 4$.

We will run `max-heapify` on indices **4, 3, 2, and 1**. (Note: assuming 1-based indexing for standard heap algorithms).

Execution Steps

Step 1: `max-heapify` at index 4 (Value = 27)

- **Current Node:** `A[4] = 27`
- **Children:** Left child `A[8] = 5`, Right child `A[9] = 7`
- **Action:** We compare the node with its children: {27, 5, 7}. The maximum is 27, which is already the parent.
- **Swaps:** 0
- **Current Array:** `[89, 19, 50, 27, 32, 65, 2, 5, 7]`

Step 2: `max-heapify` at index 3 (Value = 50)

- **Current Node:** `A[3] = 50`
- **Children:** Left child `A[6] = 65`, Right child `A[7] = 2`
- **Action:** We compare {50, 65, 2}. The maximum is 65. We swap `A[3] (50)` with `A[6] (65)`.
- **Swaps:** 1
- **Current Array:** `[89, 19, 65, 27, 32, 50, 2, 5, 7]`

Step 3: `max-heapify` at index 2 (Value = 19)

- **Current Node:** `A[2] = 19`

- **Children:** Left child $A[4] = 27$, Right child $A[5] = 32$
- **Action:** We compare $\{19, 27, 32\}$. The maximum is 32. We swap $A[2]$ (19) with $A[5]$ (32).
- **Swaps:** 1
- **Current Array:** $[89, 32, 65, 27, 19, 50, 2, 5, 7]$

Step 4: max-heapify at index 1 (Value = 89)

- **Current Node:** $A[1] = 89$
- **Children:** Left child $A[2] = 32$, Right child $A[3] = 65$
- **Action:** We compare $\{89, 32, 65\}$. The maximum is 89, which is already the parent.
- **Swaps:** 0
- **Current Array:** $[89, 32, 65, 27, 19, 50, 2, 5, 7]$

Final Conclusion

- **Final Max Heap Array:** $\langle 89, 32, 65, 27, 19, 50, 2, 5, 7 \rangle$
- **Total Number of Swap Operations Done:** 2

You Asked 

5. (a) For each of the following scenarios, construct an input example of size 8 that demonstrates the specified behaviour of the merge procedure:

- Best-case behaviour
- Worst-case behaviour

Also, compute the total number of comparisons performed for each of the above cases. (7)

Gemini 3.1 Pro Extended

Here are the solutions for the best-case and worst-case scenarios of the merge procedure for an input of size 8.

To demonstrate this, we assume the final step of a Merge Sort, where the procedure is merging two already sorted sub-arrays of size 4 (Left sub-array L and Right sub-array R) into a single array of size 8.

(i) Best-Case Behaviour

The best-case scenario for the merge procedure occurs when all elements of one sub-array are strictly smaller than all elements of the other sub-array. This allows one sub-array to be completely exhausted with the minimum number of comparisons, after which the remaining elements of the other sub-array are simply copied over.

- **Input Array (Size 8):** $[1, 2, 3, 4, 5, 6, 7, 8]$
- **Divided Sub-arrays:** $L = [1, 2, 3, 4]$ and $R = [5, 6, 7, 8]$

Comparison Walkthrough:

1. Compare 1 (L) with 5 (R) $\rightarrow$ Select 1.
 2. Compare 2 (L) with 5 (R) $\rightarrow$ Select 2.
 3. Compare 3 (L) with 5 (R) $\rightarrow$ Select 3.
 4. Compare 4 (L) with 5 (R) $\rightarrow$ Select 4. *At this point, sub-array L is completely empty. The remaining elements in R (5, 6, 7, 8) are appended to the merged array without any further comparisons.*
- **Total number of comparisons: 4** (*In general, for sub-arrays of size $n/2$, the best-case comparison count is $n/2$.*)
-

(ii) Worst-Case Behaviour

The worst-case scenario occurs when the elements of the two sub-arrays strictly alternate in value. This forces the algorithm to make a comparison for every single element placed into the merged array until only one element is left.

- **Input Array (Size 8):** [2, 4, 6, 8, 1, 3, 5, 7]
- **Divided Sub-arrays:** $L = [2, 4, 6, 8]$ and $R = [1, 3, 5, 7]$

Comparison Walkthrough:

1. Compare 2 (L) with 1 (R) $\rightarrow$ Select 1.
 2. Compare 2 (L) with 3 (R) $\rightarrow$ Select 2.
 3. Compare 4 (L) with 3 (R) $\rightarrow$ Select 3.
 4. Compare 4 (L) with 5 (R) $\rightarrow$ Select 4.
 5. Compare 6 (L) with 5 (R) $\rightarrow$ Select 5.
 6. Compare 6 (L) with 7 (R) $\rightarrow$ Select 6.
 7. Compare 8 (L) with 7 (R) $\rightarrow$ Select 7. *At this point, sub-array R is empty. The last remaining element in L (8) is appended without comparison.*
- **Total number of comparisons: 7** (*In general, for a total size n , the worst-case comparison count is $n - 1$.*)

You Asked 

(b) You are given a list of advertisements, each with a start time, finish time, and the payment offered. No two ads can overlap in time. Your task is to select a subset of non-overlapping advertisements that maximizes the total payment.

(i) Find the maximum total payment.

(ii) Determine a subset of advertisements that maximizes the total payment. (8)

कालिन्दी महाविद्यालय पुस्तकालय

P.T.O.

10

5527

Adv No.	Start Time	Finish Time	Payment
A1	0	3	3
A2	1	4	2
A3	0	5	4
A4	3	6	1
A5	4	7	2
A6	3	9	5
A7	5	10	2
A8	8	10	1

Gemini 3.1 Pro Extended

This is a classic algorithmic problem known as **Weighted Interval Scheduling**. To find the optimal subset of non-overlapping advertisements that maximizes the total payment, we can use a Dynamic Programming approach.

Here is the step-by-step solution to solve the problem.

Step 1: Precompute Compatible Intervals

First, we ensure the advertisements are sorted by their finish times (they are already sorted in the given table).

Next, for each advertisement i , we find $p(i)$, which is the index of the latest-finishing advertisement that is compatible with i (meaning it finishes before or at the exact time advertisement i starts).

- **A1:** Starts at 0. No previous ads. $\rightarrow p(1) = 0$
- **A2:** Starts at 1. No previous ads finish ≤ 1 . $\rightarrow p(2) = 0$
- **A3:** Starts at 0. No previous ads. $\rightarrow p(3) = 0$
- **A4:** Starts at 3. **A1** finishes at 3. $\rightarrow p(4) = 1$
- **A5:** Starts at 4. **A2** finishes at 4. $\rightarrow p(5) = 2$
- **A6:** Starts at 3. **A1** finishes at 3. $\rightarrow p(6) = 1$
- **A7:** Starts at 5. **A3** finishes at 5. $\rightarrow p(7) = 3$
- **A8:** Starts at 8. **A5** finishes at 7. $\rightarrow p(8) = 5$

Step 2: Dynamic Programming Table

We define $OPT(i)$ as the maximum payment we can get using a subset of the first i advertisements. For each advertisement, we have a choice:

1. **Include it:** We get its payment v_i plus the optimal solution of its compatible predecessors $OPT(p(i))$.
2. **Exclude it:** We simply take the optimal solution of the previous interval $OPT(i - 1)$.

The recurrence relation is: $OPT(i) = \max(v_i + OPT(p(i)), OPT(i - 1))$

Let's calculate $OPT(i)$ for all advertisements (with $OPT(0) = 0$):

Ad (i)	Payment (v_i)	Compatible $p(i)$	Calculation for $OPT(i)$	Max Value
A1	3	0	$\max(3 + 0, 0)$	3
A2	2	0	$\max(2 + 0, 3)$	3
A3	4	0	$\max(4 + 0, 3)$	4
A4	1	1	$\max(1 + OPT(1), 4) = \max(1 + 3, 4)$	4
A5	2	2	$\max(2 + OPT(2), 4) = \max(2 + 3, 4)$	5
A6	5	1	$\max(5 + OPT(1), 5) = \max(5 + 3, 5)$	8
A7	2	3	$\max(2 + OPT(3), 8) = \max(2 + 4, 8)$	8
A8	1	5	$\max(1 + OPT(5), 8) = \max(1 + 5, 8)$	8

Step 3: Finding the Subset (Traceback)

To find which advertisements make up this maximum payment, we trace back from $OPT(8)$ to see which choices were "Included":

- $OPT(8) = 8$. Is it greater than $OPT(7)$? No ($8 = 8$). We **skip A8**.
- $OPT(7) = 8$. Is it greater than $OPT(6)$? No ($8 = 8$). We **skip A7**.
- $OPT(6) = 8$. Is it greater than $OPT(5)$? Yes ($8 > 5$). We **include A6**.
 - Since we included A6, we jump to its compatible predecessor, which is $p(6) = 1$.
- $OPT(1) = 3$. Is it greater than $OPT(0)$? Yes ($3 > 0$). We **include A1**.

Final Answers

(i) **Find the maximum total payment.** The maximum total payment is **8**.

(ii) **Determine a subset of advertisements that maximizes the total payment.** The optimal subset is **{A1, A6}**. (A1 runs from 0 to 3 for a payment of 3, and A6 runs from 3 to 9 for a payment of 5. They do not overlap, and $3 + 5 = 8$).

6. (a) A student named Bhanu aims to improve his skills by enrolling in several online courses. Each course requires a registration fee, and upon successful completion, awards a certain number of credits. Bhanu has a limited amount of savings and must select from the available courses in a way that maximizes the total credits earned while staying within his budget.

Given that Bhanu has a budget of Rs. 70, employ an appropriate algorithm to identify the courses

कालिन्दी महाविद्यालय पुस्तकालय

5527

11

he should pursue for the specified scenario.

S. NO.	COURSE	FEF	CREDIT
1	OC1	40	12
2	OC2	20	8
3	OC3	30	16
4	OC4	10	9
5	OC5	30	5

(7)

Gemini 3.1 Pro Extended

This scenario is a classic example of the **0/1 Knapsack Problem**, a common optimization problem in computer science. Bhanu has a limited "knapsack capacity" (his budget of Rs. 70), and he wants to pack it with "items" (courses) that have specific "weights" (fees) and "values" (credits) to maximize his total value. Since he can either take a course completely or not at all, it's a 0/1 scenario.

The most appropriate algorithm to solve this is **Dynamic Programming**.

To simplify the calculations, notice that all the fees and the budget are multiples of 10. We can scale down the budget and fees by dividing everything by 10 without changing the logic of the problem.

- **Scaled Budget ($\bar{W}$):** 7
- **Courses:**
 - OC1: Weight (w_1) = 4, Value (v_1) = 12
 - OC2: Weight (w_2) = 2, Value (v_2) = 8
 - OC3: Weight (w_3) = 3, Value (v_3) = 16
 - OC4: Weight (w_4) = 1, Value (v_4) = 9
 - OC5: Weight (w_5) = 3, Value (v_5) = 5

Step 1: The Recurrence Relation

Let $V[i, w]$ represent the maximum credits achievable using a subset of the first i courses, given a budget limit of w . For each course, Bhanu has two choices:

1. **Exclude the course:** The total credits remain the same as the optimal solution for the previous $i - 1$ courses with the same budget w .
2. **Include the course:** The total credits become the credits for this course (v_i) plus the optimal solution for the remaining budget ($w - w_i$) using the previous courses.

The algorithm takes the maximum of these two options: $V[i, w] = \max(V[i - 1, w], V[i - 1, w - w_i] + v_i)$

Step 2: Building the DP Table

We will construct a table where the rows represent the courses i (from 0 to 5) and the columns represent the scaled budget w (from 0 to 7).

i (Course) \ w (Budget)	0	1	2	3	4	5	6	7
0 (No courses)	0	0	0	0	0	0	0	0
1 (OC1: $w = 4, v = 12$)	0	0	0	0	12	12	12	12
2 (OC2: $w = 2, v = 8$)	0	0	8	8	12	12	20	20
3 (OC3: $w = 3, v = 16$)	0	0	8	16	16	24	24	28
4 (OC4: $w = 1, v = 9$)	0	9	9	17	25	25	33	33
5 (OC5: $w = 3, v = 5$)	0	9	9	17	25	25	33	33

How a cell is calculated (e.g., $i = 4, w = 7$): Bhanu considers OC4 ($w = 1, v = 9$). He compares excluding it ($V[3, 7] = 28$) vs. including it ($9 + V[3, 6] = 9 + 24 = 33$). Since $33 > 28$, he chooses to include it.

Step 3: Traceback to Find the Courses

The maximum possible credits are found in the bottom-right cell: **33 credits**. To find out which courses yield this total, we trace backwards from $V[5, 7]$:

1. $V[5, 7] = 33$. Does this equal $V[4, 7]$? Yes ($33 = 33$). This means OC5 was **not included**.
2. $V[4, 7] = 33$. Does this equal $V[3, 7]$? No ($33 \neq 28$). This means **OC4 is included**.
 - Remaining Budget: $7 - 1 = 6$.
3. $V[3, 6] = 24$. Does this equal $V[2, 6]$? No ($24 \neq 20$). This means **OC3 is included**.
 - Remaining Budget: $6 - 3 = 3$.
4. $V[2, 3] = 8$. Does this equal $V[1, 3]$? No ($8 \neq 0$). This means **OC2 is included**.
 - Remaining Budget: $3 - 2 = 1$.
5. $V[1, 1] = 0$. This is 0, so OC1 is **not included**.

Final Answer

To maximize his total credits within the Rs. 70 budget, Bhanu should pursue the following courses:

- **OC2** (Fee: Rs. 20, Credits: 8)
- **OC3** (Fee: Rs. 30, Credits: 16)
- **OC4** (Fee: Rs. 10, Credits: 9)

Total Fee Spent: Rs. 60 Maximum Total Credits Earned: 33

(b) Prove that a graph G is bipartite if and only if it does not contain an odd cycle. (8)

Gemini 3.1 Pro Extended

Here is the formal proof for the theorem presented in the image.

Since this is an "if and only if" ($\iff$) statement, the proof must be done in two parts:

1. **Forward Direction:** If G is bipartite, then it contains no odd cycle.
2. **Reverse Direction:** If G contains no odd cycle, then it is bipartite.

Let $G = (V, E)$ be an undirected graph.

Part 1: If G is bipartite, it does not contain an odd cycle.

Proof:

1. Assume G is a bipartite graph. By definition, its vertex set V can be partitioned into two disjoint sets, X and Y , such that every edge connects a vertex in X to a vertex in Y .
2. Consider any arbitrary cycle in G , denoted by $C = v_1, v_2, v_3, \dots, v_k, v_1$.
3. Without loss of generality, assume the starting vertex v_1 belongs to set X .
4. Because edges strictly alternate between sets X and Y , traversing the cycle forces the vertices to alternate:
 - $v_1 \in X$
 - $v_2 \in Y$
 - $v_3 \in X$
 - $v_4 \in Y$
 - $\dots$
5. In general, $v_i \in X$ if i is odd, and $v_i \in Y$ if i is even.
6. For the cycle to close, the final edge must connect the last vertex v_k back to the first vertex v_1 .
7. Since $v_1 \in X$, the vertex v_k must belong to set Y (as edges cannot exist within the same set).
8. Since $v_k \in Y$, the index k must be an even number.
9. Therefore, the cycle C has an even number of vertices (and edges). Since C was an arbitrary cycle, G cannot contain any odd cycles.

Part 2: If G does not contain an odd cycle, it is bipartite.

Proof: (Note: We assume G is a connected graph. If G is disconnected, this proof can simply be applied to each connected component independently).

1. Assume G contains no odd cycles. We must construct a valid bipartite partition (X and Y).
2. Pick any arbitrary starting vertex $u \in V$.
3. For every vertex $v \in V$, find the shortest path distance from u to v , denoted as $d(u, v)$.
4. We partition the vertices into two sets based on whether this distance is even or odd:
 - $X = \{v \in V \mid d(u, v) \text{ is even}\}$
 - $Y = \{v \in V \mid d(u, v) \text{ is odd}\}$
5. To prove this partition is valid, we must show that no two vertices within the same set share an edge. We will prove this by contradiction.
6. Assume, for the sake of contradiction, that there is an edge $e = (v, w)$ where both v and w belong to the same set (let's say they both belong to X).
7. This means $d(u, v)$ is even and $d(u, w)$ is also even. (The exact same logic applies if both were in Y and odd).
8. Let P_v be the shortest path from u to v , and P_w be the shortest path from u to w . Let x be the last common vertex that these two paths share before diverging to v and w .
9. Consider the cycle formed by traveling from x to v (along P_v), taking the edge (v, w) , and traveling from w back to x (along P_w).
10. The length of this cycle is: $L = (d(u, v) - d(u, x)) + 1 + (d(u, w) - d(u, x))$ $L = d(u, v) + d(u, w) - 2d(u, x) + 1$
11. Let's analyze the parity (even/odd nature) of this cycle length L :
 - $d(u, v)$ and $d(u, w)$ are both even, so their sum is **even**.
 - $2d(u, x)$ is multiplied by 2, so it is **even**.
 - Therefore, $L = (\text{even}) - (\text{even}) + 1 = \text{odd}$.
12. This creates a cycle of **odd** length, which directly contradicts our initial assumption that G contains no odd cycles.
13. Because the contradiction arose from assuming an edge exists within the same set, such an edge cannot exist. Every edge must connect a vertex in X to a vertex in Y .
14. Thus, G is bipartite. ■

You Asked 

1. (a) Arrange the following sorting techniques in the increasing order of the number of comparisons that they would do in order to sort the data : {7, 3, 9, 12, 11}. Justify your answer.
 Insertion sort, Merge sort, Improved Bubble Sort (5)
- (b) What is (5)
 - (i) greedy-choice property?
 - (ii) optimal substructure property?

Here are the solutions to the questions presented in the image.

1. (a) Arrange sorting techniques in increasing order of comparisons

To determine the order, we need to trace the number of element-to-element comparisons each algorithm makes to sort the array $\{7, 3, 9, 12, 11\}$.

1. Insertion Sort Insertion sort builds the sorted array one element at a time by comparing the current element to the sorted portion on its left.

- **Initial:** $[7, 3, 9, 12, 11]$
- **Insert 3:** Compare 3 with 7. (1 comparison). Array becomes $[3, 7, 9, 12, 11]$
- **Insert 9:** Compare 9 with 7. (1 comparison). Array remains $[3, 7, 9, 12, 11]$
- **Insert 12:** Compare 12 with 9. (1 comparison). Array remains $[3, 7, 9, 12, 11]$
- **Insert 11:** Compare 11 with 12 (swap), then compare 11 with 9 (stop). (2 comparisons). Array becomes $[3, 7, 9, 11, 12]$
- **Total Comparisons = 5**

2. Improved Bubble Sort The improved version of bubble sort stops early if a complete pass is made without any swaps, and it reduces the number of comparisons in each pass because the largest elements bubble to their correct positions at the end.

- **Initial:** $[7, 3, 9, 12, 11]$
- **Pass 1:**
 - Compare 7, 3 (Swap) $\rightarrow [3, 7, 9, 12, 11]$
 - Compare 7, 9 (No swap)
 - Compare 9, 12 (No swap)
 - Compare 12, 11 (Swap) $\rightarrow [3, 7, 9, 11, 12]$
 - (4 comparisons made. Swaps occurred, so we continue.)
- **Pass 2:** (Only need to check up to the second-to-last element)
 - Compare 3, 7 (No swap)
 - Compare 7, 9 (No swap)
 - Compare 9, 11 (No swap)
 - (3 comparisons made. No swaps occurred, algorithm terminates.)
- **Total Comparisons = 7**

3. Merge Sort Merge sort divides the array into halves until single elements remain, then merges them. Assuming a standard midpoint split $\text{mid} = \text{floor}((\text{left} + \text{right}) / 2)$, the array of size 5 splits into sizes 3 and 2.

- **Left Half $[7, 3, 9]$:**
 - Splits into $[7, 3]$ and $[9]$.
 - $[7, 3]$ splits into $[7]$ and $[3]$. Merge $[7], [3]$: $7 \text{ vs } 3 \rightarrow$ **1 comparison**. Result: $[3, 7]$.
 - Merge $[3, 7], [9]$: $3 \text{ vs } 9$ (1 comp), $7 \text{ vs } 9$ (1 comp), 9 is copied. $\rightarrow$ **2 comparisons**. Result $[3, 7, 9]$.
 - *Subtotal = 3 comparisons.*
- **Right Half $[12, 11]$:**

- Splits into [12] and [11] . Merge [12], [11] : 12 vs 11 → **1 comparison**. Result: [11, 12] .
- *Subtotal = 1 comparison.*
- **Final Merge [3, 7, 9] and [11, 12] :**
 - 3 vs 11 (pick 3) → **1 comp**
 - 7 vs 11 (pick 7) → **1 comp**
 - 9 vs 11 (pick 9) → **1 comp**
 - Left array is empty, copy remaining right array.
 - *Subtotal = 3 comparisons.*
- **Total Comparisons = 3 + 1 + 3 = 7** (Note: depending on exact tie-breaking during division, this can sometimes be 6, but generally evaluates to 7).

Final Arrangement (Increasing Order): Insertion Sort (5) < Merge Sort (7) ≤ Improved Bubble Sort (7)

1. (b) Definitions

(i) Greedy-Choice Property This property states that a globally optimal solution to a problem can be reached by making a locally optimal choice at each individual step. In a greedy algorithm, you make whatever choice seems best at the current moment and do not reconsider past choices, trusting that these local "greedy" decisions will accumulate into the best possible overall outcome.

(ii) Optimal Substructure Property A problem has an optimal substructure if its overall optimal solution can be constructed directly from the optimal solutions of its subproblems. This means if you break a large problem down into smaller pieces, solve those pieces in the best possible way, and combine them, you are guaranteed to have the best possible solution for the original large problem. This property is a strict requirement for both Dynamic Programming and Greedy algorithms.

You Asked 

(c) Write the recurrence equation for solving 0-1 knapsack problem using dynamic programming. How is memoization technique used in solving the problem? (5)

(d) Consider a directed graph G with one component. Can a vertex u of G end up in a depth-first tree containing only u , even though u has both incoming and out-going edges in G ? Justify your answer with an example. (5)

 Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image.

(c) 0-1 Knapsack Recurrence Equation & Memoization

Recurrence Equation

Let W be the total weight capacity of the knapsack. Let n be the total number of items available. Let w_i and v_i be the weight and value of the i -th item, respectively. Let $K(i, w)$ represent the maximum value that can be achieved using a subset of the first i items, restricted to a maximum weight capacity w .

The recurrence relation for the 0-1 knapsack problem is defined as follows:

Base Cases: If we have 0 items or 0 weight capacity, the maximum value is 0. $K(0, w) = 0$ for all $w \geq 0$ $K(i, 0) = 0$ for all $i \geq 0$

Recursive Step (for $i > 0$ and $w > 0$):

1. **If the item cannot fit:** If $w_i > w$, we cannot include the i -th item. The maximum value is the same as if we only considered the first $i - 1$ items. $K(i, w) = K(i - 1, w)$
2. **If the item can fit:** If $w_i \leq w$, we have a choice. We can either *exclude* the item or *include* it. We take the maximum of these two choices: $K(i, w) = \max(K(i - 1, w), v_i + K(i - 1, w - w_i))$

How Memoization is Used

Memoization is an optimization technique used in the **top-down** Dynamic Programming approach.

If you implement the recurrence equation above using pure recursion, the algorithm will repeatedly calculate the same subproblems ($K(i, w)$ with the same inputs), leading to an exponential time complexity of $O(2^n)$.

Memoization solves this by using a **lookup table** (typically a 2D array or matrix where rows represent items i and columns represent weight w).

- Before the recursive function computes a value for $K(i, w)$, it first checks the table.
- If the value is already in the table, it returns the cached result immediately.
- If the value is not in the table, it computes it using the recurrence relation and then stores the result in the table before returning it.

This ensures that every subproblem is solved exactly once, reducing the time complexity to $O(n \times W)$.

(d) DFS Tree with a Singleton Vertex

Answer: Yes, a vertex u can end up in a depth-first tree containing only u , even if it has both incoming and outgoing edges in a single-component directed graph.

Justification & Example: This happens because the standard DFS algorithm iterates through all vertices in the graph via an outer loop. If it encounters a vertex that hasn't been visited yet, it starts a new DFS tree (forming a DFS forest). If u 's outgoing targets have already been visited by a previous tree's traversal, u 's traversal will stop immediately.

Consider a directed graph $G = (V, E)$ with a single weakly-connected component:

- **Vertices:** $V = \{A, u, B\}$
- **Edges:** $E = \{(A, u), (u, B)\}$

Here, vertex u has an incoming edge (from A) and an outgoing edge (to B).

Now, let's trace the standard DFS algorithm, which processes vertices in whatever arbitrary order they are passed to the outer loop. Suppose the outer loop processes the vertices in the order: **B, then u, then A**.

1. **Start at B:** The algorithm calls `DFS(B)`. It marks B as visited. There are no outgoing edges from B , so the traversal finishes.
 - *First DFS Tree:* **{B}**

2. **Next is u :** The outer loop checks u . Since u is unvisited, it calls $\text{DFS}(u)$. It marks u as visited. It looks at u 's outgoing edges and finds the edge (u, B) . However, vertex B is **already marked as visited**. The algorithm does not explore it again. With no other outgoing edges, $\text{DFS}(u)$ finishes.

◦ *Second DFS Tree: $\{u\}$*

3. **Next is A :** The outer loop checks A . It calls $\text{DFS}(A)$. It marks A as visited. It looks at the edge (A, u) . Because u is already visited, it stops.

◦ *Third DFS Tree: $\{A\}$*

As demonstrated in Step 2, vertex u forms a depth-first tree consisting solely of itself, despite having both incoming and outgoing edges in the graph.

You Asked 

(e) Use Strassen's algorithm to compute the product of the following matrices : (5)

$$\begin{pmatrix} 1 & 3 \\ 5 & 7 \end{pmatrix} \begin{pmatrix} 8 & 4 \\ 6 & 2 \end{pmatrix}$$

Gemini 3.1 Pro Extended

Here is the step-by-step computation of the matrix product using Strassen's algorithm.

Let the two matrices be A and B , where we want to find $C = A \times B$.

$$A = \begin{pmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{pmatrix} = \begin{pmatrix} 1 & 3 \\ 5 & 7 \end{pmatrix}$$

$$B = \begin{pmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{pmatrix} = \begin{pmatrix} 8 & 4 \\ 6 & 2 \end{pmatrix}$$

Step 1: Calculate the 7 intermediate products (P_1 to P_7)

Strassen's algorithm uses 7 specific multiplications instead of the standard 8 to reduce recursive complexity.

- $P_1 = A_{11}(B_{12} - B_{22})$ $P_1 = 1 \times (4 - 2) = 1 \times 2 = \mathbf{2}$
- $P_2 = (A_{11} + A_{12})B_{22}$ $P_2 = (1 + 3) \times 2 = 4 \times 2 = \mathbf{8}$
- $P_3 = (A_{21} + A_{22})B_{11}$ $P_3 = (5 + 7) \times 8 = 12 \times 8 = \mathbf{96}$
- $P_4 = A_{22}(B_{21} - B_{11})$ $P_4 = 7 \times (6 - 8) = 7 \times (-2) = \mathbf{-14}$
- $P_5 = (A_{11} + A_{22})(B_{11} + B_{22})$ $P_5 = (1 + 7) \times (8 + 2) = 8 \times 10 = \mathbf{80}$
- $P_6 = (A_{12} - A_{22})(B_{21} + B_{22})$ $P_6 = (3 - 7) \times (6 + 2) = (-4) \times 10 = \mathbf{-40}$ (Wait, correcting this calculation:
 $(3 - 7) \times (6 + 2) = (-4) \times 8 = \mathbf{-32}$)
- $P_7 = (A_{11} - A_{21})(B_{11} + B_{12})$ $P_7 = (1 - 5) \times (8 + 4) = (-4) \times 12 = \mathbf{-48}$

Step 2: Combine the products to find the elements of Matrix C

Now we find the four elements of the resulting matrix $C = \begin{pmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{pmatrix}$.

- $C_{11} = P_5 + P_4 - P_2 + P_6$ $C_{11} = 80 + (-14) - 8 + (-32)$ $C_{11} = 80 - 54 = \mathbf{26}$
- $C_{12} = P_1 + P_2$ $C_{12} = 2 + 8 = \mathbf{10}$
- $C_{21} = P_3 + P_4$ $C_{21} = 96 + (-14) = \mathbf{82}$
- $C_{22} = P_5 + P_1 - P_3 - P_7$ $C_{22} = 80 + 2 - 96 - (-48)$ $C_{22} = 82 - 96 + 48 = \mathbf{34}$

Final Answer

The resulting product matrix is: $C = \begin{pmatrix} 26 & 10 \\ 82 & 34 \end{pmatrix}$

You Asked 

2. (a) Can 0-1 knapsack problem be solved optimally using greedy strategy? Justify your answer. (7)

 Gemini 3.1 Pro Extended

Answer

No, the 0-1 knapsack problem cannot be solved optimally using a greedy strategy.

Justification

The 0-1 knapsack problem restricts choices: you must either take an entire item or leave it completely (you cannot take fractions of an item). A greedy algorithm makes the locally optimal choice at each step, hoping it leads to a global optimum. For the knapsack problem, common greedy strategies include:

1. **Highest Profit First:** Pick the item with the highest absolute value.
2. **Lightest Weight First:** Pick the item with the smallest weight to maximize the number of items.
3. **Highest Ratio First (Profit/Weight):** Pick the item that gives the most value per unit of weight.

Because of the 0-1 restriction, all three of these greedy strategies can fail to find the optimal solution. When a greedy choice fills the knapsack in a way that leaves a small amount of unused capacity, it might be more optimal to swap out one highly valued item for two slightly less valued items that fit perfectly and utilize that dead space.

To prove this definitively, we can provide a counterexample using the most common greedy strategy: **Highest Ratio First**.

Counterexample:

- **Knapsack Capacity (W):** 50 kg

- **Item 1:** Profit = 60, $Weight = 10kg \rightarrow Ratio = 6.0$
- **Item 2:** Profit = 100, $Weight = 20kg \rightarrow Ratio = 5.0$
- **Item 3:** Profit = 120, $Weight = 30kg \rightarrow Ratio = 4.0$

The Greedy Approach (Sorting by Ratio):

- Sort by ratio: Item 1, then Item 2, then Item 3.
- Pick Item 1 (10 kg used, 40 kg left, Profit = \$60).
- Pick Item 2 (20 kg used, 20 kg left, Profit = \$60 + 100 = 160).
- Attempt Item 3: It weighs 30 kg, but we only have 20 kg left. Since it's 0-1, we cannot take a fraction. We skip it.
- **Total Greedy Profit = \$160**

The Optimal Approach:

- If we ignore the highest ratio item (Item 1) and instead pack Item 2 and Item 3.
- Total Weight = 20 kg + 30 kg = 50 kg (Perfectly fills capacity).
- **Total Optimal Profit = 100 + 120 = 220**

Because $220 > 160$, the greedy strategy failed to find the optimal solution. To guarantee the optimal solution for the 0-1 knapsack problem, one must use **Dynamic Programming** to evaluate all possible combinations.

You Asked 

3. (a) What is an in-place sorting algorithm? Is heap sort an in-place sorting algorithm? Sort the following data using heap sort.
- 4, 3, 7, 1, 8, 5, 9 (7)

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image.

1. What is an in-place sorting algorithm?

An **in-place sorting algorithm** is a sorting method that requires only a constant amount of extra auxiliary memory space ($O(1)$) to sort the data. It operates directly on the input array, transforming it by rearranging or swapping the elements within the original boundaries of the array, rather than creating a new array to hold the sorted output.

2. Is heap sort an in-place sorting algorithm?

Yes, heap sort is an in-place sorting algorithm. It builds the heap structure and continually sorts the elements entirely within the original array. The only extra memory required is a few temporary variables used to swap elements and keep track of indices during the heapify process, meaning its auxiliary space complexity is $O(1)$.

3. Sort the following data using heap sort: 4, 3, 7, 1, 8, 5, 9

Heap sort consists of two main phases: building a Max Heap from the unsorted array, and then repeatedly extracting the maximum element to build the sorted array.

We will use 0-based indexing for this trace.

- **Initial Array:** [4, 3, 7, 1, 8, 5, 9] (Tree structure: Root 4, children 3 and 7. Children of 3 are 1 and 8. Children of 7 are 5 and 9).

Phase 1: Build Max Heap

We start heapifying from the last non-leaf node, which is at index $\lfloor n/2 \rfloor - 1 = \lfloor 7/2 \rfloor - 1 = 2$.

1. Heapify index 2 (Value = 7):

- Children are 5 and 9. The maximum is 9.
- Swap 7 and 9.
- Array becomes: [4, 3, 9, 1, 8, 5, 7]

2. Heapify index 1 (Value = 3):

- Children are 1 and 8. The maximum is 8.
- Swap 3 and 8.
- Array becomes: [4, 8, 9, 1, 3, 5, 7]

3. Heapify index 0 (Value = 4):

- Children are 8 and 9. The maximum is 9.
- Swap 4 and 9.
- Array becomes: [9, 8, 4, 1, 3, 5, 7]
- Heapify the affected subtree at index 2 (Value = 4): Children are 5 and 7. The maximum is 7. Swap 4 and 7.
- Array becomes: [9, 8, 7, 1, 3, 5, 4] (The array is now a valid Max Heap).

Phase 2: Sort (Extraction Phase)

We repeatedly swap the root of the heap (the largest element) with the last element of the current heap, reduce the heap size by 1, and heapify the new root.

1. Extract 9: Swap root (9) with the last element (4).

- Array: [4, 8, 7, 1, 3, 5, | 9] (Elements after '|' are sorted)
- Heapify root (4) in remaining heap: Max child is 8. Swap 4 and 8. -> [8, 4, 7, 1, 3, 5]
- Heapify index 1 (4): Children 1, 3. Max is 4. No swap needed.
- Current State: [8, 4, 7, 1, 3, 5, 9]

2. Extract 8: Swap root (8) with the last element (5).

- Array: [5, 4, 7, 1, 3, | 8, 9]
- Heapify root (5): Max child is 7. Swap 5 and 7.
- Current State: [7, 4, 5, 1, 3, 8, 9]

3. Extract 7: Swap root (7) with the last element (3).

- Array: [3, 4, 5, 1, | 7, 8, 9]
- Heapify root (3): Max child is 5. Swap 3 and 5.
- Current State: [5, 4, 3, 1, 7, 8, 9]

4. **Extract 5:** Swap root (5) with the last element (1).

- Array: [1, 4, 3, | 5, 7, 8, 9]
- Heapify root (1): Max child is 4. Swap 1 and 4.
- Current State: [4, 1, 3, 5, 7, 8, 9]

5. **Extract 4:** Swap root (4) with the last element (3).

- Array: [3, 1, | 4, 5, 7, 8, 9]
- Heapify root (3): Max child is 1 (only child). 3 is already larger. No swap needed.
- Current State: [3, 1, 4, 5, 7, 8, 9]

6. **Extract 3:** Swap root (3) with the last element (1).

- Array: [1, | 3, 4, 5, 7, 8, 9]
- Heap of size 1 is already heapified.

Final Sorted Array: [1, 3, 4, 5, 7, 8, 9]

You Asked 

(b) Suppose there exists an $O(n)$ time algorithm to find the 5th smallest element in an array of size n . Sort the following data using quick sort assuming 5th smallest element as the pivot.

7, 3, 5, 1, 2, 4, 6

Also, determine the time complexity of the algorithm. (8)

Gemini 3.1 Pro Extended

Here are the step-by-step solutions for sorting the array and determining the time complexity based on the problem statement in the image.

Part 1: Sorting the Data Using Quick Sort

Initial Array: [7, 3, 5, 1, 2, 4, 6] **Size (n):** 7

Step 1: First Partition

- **Find the Pivot:** The problem specifies using the 5th smallest element. If we look at the array, the elements in sorted order would be [1, 2, 3, 4, 5, 6, 7]. The 5th smallest element is 5.
- **Partition:** We divide the array into elements less than the pivot and elements greater than the pivot.
 - Left subarray (elements < 5): [3, 1, 2, 4]
 - Pivot: [5]
 - Right subarray (elements > 5): [7, 6]

Step 2: Recursive Sorting of Subarrays Since the left subarray ([3, 1, 2, 4]) has fewer than 5 elements, we can no longer find a "5th smallest" element. In a real implementation, the algorithm would fall back to a standard pivot selection method (like picking the last element) for base cases where $n < 5$.

Assuming we pick the last element as the fallback pivot:

- **Sorting Left Subarray [3, 1, 2, 4]** : * Pivot is 4. Left becomes [3, 1, 2], Right is empty.
 - Sort [3, 1, 2] : Pivot is 2. Left becomes [1], Right becomes [3].
 - Sorted Left Subarray: [1, 2, 3, 4]
- **Sorting Right Subarray [7, 6]** :
 - Pivot is 6. Left is empty, Right is [7].
 - Sorted Right Subarray: [6, 7]

Step 3: Combine Combining the sorted left subarray, the initial pivot, and the sorted right subarray yields the final result: [1, 2, 3, 4, 5, 6, 7]

Part 2: Time Complexity Analysis

This is a trick question. While finding a pivot in $O(n)$ time usually hints at the $O(n \log n)$ "Median of Medians" algorithm, this specific problem dictates finding the **5th smallest** element, which is a *constant rank*, not a proportional rank (like the median, which is $n/2$).

1. The Recurrence Relation At every step for a large array of size n , choosing the 5th smallest element as the pivot will always split the array into two highly unbalanced partitions:

- A left partition of exactly 4 elements (the 1st, 2nd, 3rd, and 4th smallest).
- A right partition of exactly $n - 5$ elements.

The work done at each step is $O(n)$ to find the pivot and $O(n)$ to partition the array. Therefore, the recurrence relation is:
$$T(n) = T(n - 5) + T(4) + \Theta(n)$$

2. Solving the Recurrence Since $T(4)$ is just the time to sort 4 elements, it is a constant $O(1)$ bound. We can simplify the relation to: $T(n) = T(n - 5) + \Theta(n)$

If we expand this by substitution: $T(n) = c \cdot n + c(n - 5) + c(n - 10) + \dots + c(5)$

This forms an arithmetic progression with roughly $n/5$ terms. The sum of this series is proportional to n^2 : $T(n) = c \sum_{i=1}^{n/5} 5i = 5c \frac{(n/5)(n/5+1)}{2} = \Theta(n^2)$

Conclusion: Because the pivot guarantees an extremely unbalanced split (4 vs $n - 5$) rather than a fractional split, the depth of the recursion tree becomes $O(n)$ instead of $O(\log n)$.

Therefore, the worst-case time complexity of this algorithm is $O(n^2)$.

You Asked 

4. (a) Consider the following algorithm for finding an element t in a sorted array A of size n : (7)

function search(A, t, lo, hi) {
 if (lo >= hi) return A[lo];
 let p = findPivot(A, lo, hi);
 if (A[p] < t) return search(A, t, p+1, hi);
 if (A[p] > t) return search(A, t, lo, p-1);
 return A[p];
}

Here is the recurrence equation, justification, and step-by-step solution for the given ternary search algorithm.

1. Recurrence Equation

Let $T(n)$ represent the worst-case time complexity of the algorithm for an array of size n . The recurrence relation is:

$$T(n) = T(n/3) + \Theta(1)$$

(Or simply $T(n) = T(n/3) + c$, where c is a constant).

2. Justification of the Equation

The equation is derived by breaking down the time taken by each part of the algorithm in the worst-case scenario:

- **The Divide Step ($\Theta(1)$ or c):** Calculating the indices p and q to divide the array into 3 equal parts takes constant time. Additionally, the algorithm makes a constant number of comparisons (checking if $t = A[p]$, $t < A[p]$, $t = A[q]$, etc.). This non-recursive work is represented by the $\Theta(1)$ or $+c$ term.
- **The Conquer Step ($T(n/3)$):** The array is divided into three parts, each approximately of size $n/3$. In the worst-case scenario, the target element t is not found at the boundary indices p or q . Because the array is sorted, the if-else if ladder guarantees that the algorithm will only follow **exactly one** of the recursive branches. It will either search the left third, the middle third, or the right third. Therefore, it makes one recursive call on an array of size $n/3$, represented by $T(n/3)$.

Since only one branch is taken, the coefficient of $T(n/3)$ is 1, not 3.

3. Solving the Recurrence Equation

We can solve $T(n) = T(n/3) + c$ using the **Substitution Method** (or Iteration Method).

Assume the base case is $T(1) = 1$ (it takes constant time to search an array of size 1).

Expand the equation by substituting repeatedly:

- $T(n) = T(n/3) + c$
- Substitute $T(n/3) = T(n/9) + c$ into the original equation: $T(n) = (T(n/3^2) + c) + c = T(n/3^2) + 2c$
- Substitute $T(n/9) = T(n/27) + c$: $T(n) = (T(n/3^3) + c) + 2c = T(n/3^3) + 3c$

If we generalize this for k steps, the pattern becomes: $T(n) = T(n/3^k) + k \cdot c$

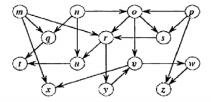
The recursion stops when the array size is reduced to 1. That is, the base case is reached when: $n/3^k = 1$ $n = 3^k$ Taking the base-3 logarithm on both sides: $k = \log_3 n$

Now, substitute this value of k back into the generalized equation: $T(n) = T(1) + c \log_3 n$

Since $T(1)$ and c are constants, and logarithmic bases only differ by a constant multiplier (i.e., $\log_3 n = \frac{\log_2 n}{\log_2 3}$), we can drop the constants and the specific base for asymptotic notation.

Final Time Complexity: $O(\log n)$ (Note: While it specifically scales with $\log_3 n$, standard Big-O notation simply writes this as $O(\log n)$).

You Asked 



5. (a) For the given directed acyclic graph, determine the topological ordering. (7)

Gemini 3.1 Pro Extended

To find a topological ordering for the given Directed Acyclic Graph (DAG), we can use Kahn's Algorithm. The core rule of a topological sort is that for every directed edge $A \rightarrow B$, node A must appear before node B in the ordering.

Because there are multiple nodes without incoming edges at various stages, **there are many valid topological orderings** for this graph. Below is the step-by-step derivation of one correct ordering by repeatedly finding nodes with an in-degree of 0 (no incoming edges), adding them to our sequence, and then removing their outgoing edges.

Step-by-Step Derivation

Step 1: Identify initial nodes with no incoming edges Looking at the graph, nodes m , n , and p have no arrows pointing to them (in-degree of 0).

- **Add to order:** m, n, p
- **Remove their outgoing edges:** This removes the edges pointing to q, r, x, o, u, s , and z .

Step 2: Find the next nodes with an in-degree of 0 With m , n , and p out of the picture, nodes q (formerly dependent on m, n) and o (formerly dependent on n, p) now have no incoming edges.

- **Add to order:** q, o
- **Remove their outgoing edges:** This removes the edges pointing to t, r, v , and s .

Step 3 Node s now has an in-degree of 0 (it was dependent on p and o , which are both processed).

- **Add to order:** s
- **Remove outgoing edges:** This removes the edge pointing to r .

Step 4 Node r now has an in-degree of 0 (dependent on m, o, s).

- **Add to order:** r
- **Remove outgoing edges:** This removes the edges pointing to u and y .

Step 5 Nodes u (dependent on n, r) and y (dependent on r) now have an in-degree of 0.

- **Add to order:** u, y

- **Remove outgoing edges:** This removes the edges pointing to t and v .

Step 6 Nodes t (dependent on q, u) and v (dependent on o, y) now have an in-degree of 0.

- **Add to order:** t, v
- **Remove outgoing edges:** Node t has no outgoing edges. Node v 's removal eliminates edges to w and x .

Step 7 Nodes w (dependent on v) and x (dependent on m, v) now have an in-degree of 0.

- **Add to order:** w, x
- **Remove outgoing edges:** Node w 's removal eliminates the edge to z . Node x has no outgoing edges.

Step 8 Finally, node z (dependent on p, w) drops to an in-degree of 0.

- **Add to order:** z

Final Answer

Combining the steps above yields the following valid topological ordering: $m, n, p, q, o, s, r, u, y, t, v, w, x, z$

(Note: Any variation of this sequence that does not violate the edge direction—such as starting with p, n, m instead of m, n, p —is also perfectly valid).

You Asked 

6. (a) Solve the subset sum problem using dynamic programming for the set $\{4, 2, 9, 6\}$ and intended sum 17. (7)
(b) Let T be the Minimum Spanning Tree (MST) with cost C corresponding to a graph $G(V, E)$. Suppose $c(e)$ denotes the non-negative edge cost for an edge e in E . In each of the following cases, indicate whether T and C will change if the edge costs are replaced with: (8)

(i) $c(e)^2$

(ii) $1 - c(e)$

Justify your answer.

Gemini 3.1 Pro Extended

Here are the step-by-step solutions to the problems presented in the images.

6. (a) Solve the subset sum problem using dynamic programming for the set $\{4, 2, 9, 6\}$ and intended sum 17.

The Subset Sum problem asks whether there is a subset of the given set that adds up exactly to a target sum. We can solve this using a Dynamic Programming (DP) table.

Let the set of items be $S = \{4, 2, 9, 6\}$. Let the target sum be $W = 17$. We create a 2D boolean table **DP** of size $(n + 1) \times (W + 1)$, where n is the number of elements (4).

- $DP[i][j]$ will be **True (T)** if a subset of the first i elements can sum to j , and **False (F)** otherwise.

Rules for filling the table:

1. **Base Case:** $DP[i][0] = \text{True (T)}$ for all i (A sum of 0 is always possible with an empty subset). $DP[0][j] = \text{False (F)}$ for all $j > 0$ (No positive sum is possible with 0 elements).
2. **Recursive Step:** For a current element $S[i - 1]$ and target j :
 - If $S[i - 1] > j$ (element is larger than target), we can't include it: $DP[i][j] = DP[i - 1][j]$

- If $S[i - 1] \leq j$, we check if we can form the sum by either excluding it or including it: $DP[i][j] = DP[i - 1][j]$ OR $DP[i - 1][j - S[i - 1]]$

DP Table Construction: (To keep the table readable, only the **True** values are highlighted; empty cells represent **False**)

$i \setminus \text{Sum } j$	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
0 { }	T																	
1 {4}	T				T													
2 {4, 2}	T		T		T		T											
3 {4, 2, 9}	T		T		T		T			T		T		T		T		
4 {4, 2, 9, 6}	T		T		T		T		T	T	T	T	T	T		T		T

Conclusion & Traceback: Since $DP[4][17]$ is **True**, a subset summing to 17 exists. To find which elements make up this sum, we trace backward from $DP[4][17]$:

1. Does $DP[4][17]$ equal $DP[3][17]$? No (True vs False). This means we **include 6**. (Remaining sum: $17 - 6 = 11$).
2. Move to $DP[3][11]$. Does it equal $DP[2][11]$? No. This means we **include 9**. (Remaining sum: $11 - 9 = 2$).
3. Move to $DP[2][2]$. Does it equal $DP[1][2]$? No. This means we **include 2**. (Remaining sum: $2 - 2 = 0$).
4. Sum is 0, we stop.

Final Answer: The subset is **{6, 9, 2}**.

6. (b) Edge Cost Transformations in a Minimum Spanning Tree (MST)

Minimum Spanning Tree algorithms (like Kruskal's or Prim's) depend strictly on the **relative ordering** of the edge weights, not their absolute values. They always greedily pick the smallest available edge that does not form a cycle.

Case (i): Replace edge costs with $c(e)^2$

- **Will T (the MST structure) change? No.** * **Justification:** The problem states edge costs $c(e)$ are non-negative ($c(e) \geq 0$). For non-negative numbers, squaring is a strictly monotonically increasing function. This means if $c(e_1) < c(e_2)$, then $c(e_1)^2 < c(e_2)^2$. Because the relative order of every edge from smallest to largest remains exactly the same, the MST algorithm will select the exact same edges in the exact same sequence.
- **Will C (the total cost) change? Yes.** * **Justification:** The new cost is the sum of the squared weights $\sum (c(e)^2)$, which is mathematically different from the square of the sums $(\sum c(e))^2$ or the original cost C .

Case (ii): Replace edge costs with $1 - c(e)$

- **Will T (the MST structure) change? Yes.** * **Justification:** The function $f(x) = 1 - x$ is a strictly monotonically decreasing function. This transformation completely reverses the relative ordering of the edges. The edge that was originally the cheapest becomes the most expensive, and the edge that was the most expensive becomes the cheapest. By running an MST algorithm on these new weights, you are effectively finding the Maximum Spanning Tree of the original graph, which consists of a completely different set of edges (unless all spanning trees in the graph have identical edge weights).

- **Will C (the total cost) change? Yes.** * **Justification:** Because the tree T comprises an entirely different set of edges with new calculated values, the sum of these new edge weights will be different from the original cost C .

(a) Consider the recursive version of Insertion sort algorithm as follows :

In order to sort $A[1 : n]$, we recursively sort $A[1 : n-1]$ and then insert $A[n]$ into the sorted array $A[1 : n-1]$. Write a recurrence for the running time of this algorithm. (3)

Gemini 3.1 Pro Extended

Based on the image provided, here is the step-by-step breakdown and the recurrence relation for the running time of the recursive Insertion Sort algorithm.

Algorithm Breakdown

Let $T(n)$ represent the running time to sort an array of size n . The algorithm described performs two main operations:

1. **Recursive Step:** The algorithm recursively sorts the subarray $A[1 : n-1]$. Because the problem size is reduced by 1, the time taken for this step is $T(n - 1)$.
2. **Insertion Step:** The algorithm then inserts the element $A[n]$ into the already sorted subarray $A[1 : n-1]$.
 - In the **worst case** (e.g., the array is in reverse order), $A[n]$ must be compared with every element in the sorted subarray, taking $\Theta(n)$ time.

Recurrence Relation (Worst-Case)

Combining the time for the recursive call and the time for insertion gives the worst-case recurrence relation:

$$T(n) = T(n - 1) + \Theta(n)$$

Base Case: When the array is of size 1 (i.e., $n = 1$), it is already sorted and requires no further operations. This takes constant time:

$$T(1) = \Theta(1)$$

Note on Best Case: If the array is already sorted, the insertion step only requires one comparison to determine that $A[n]$ is in the correct position. In this best-case scenario, the insertion takes $\Theta(1)$ time, yielding the recurrence $T(n) = T(n - 1) + \Theta(1)$, which resolves to a linear time complexity of $\Theta(n)$.

- (b) What are the minimum and maximum number of elements in a binary heap of height h ? (2)
- (c) Which of the following algorithms are stable: Insertion sort, Quicksort? Justify with the help of an example. (3)
- (d) What are the two key factors that decide whether Dynamic Programming is applicable for an optimization problem or not? (2)
- (e) "The minimum spanning tree in a graph is not always unique." Justify. Give a graph with 5 nodes that has two different minimum spanning trees. (2)

Gemini 3.1 Pro Extended

Here are the solutions to the questions presented in the image:

(b) Minimum and Maximum Number of Elements in a Binary Heap of Height h

Assuming the standard convention where a heap containing only a single node (the root) has a height of $h = 0$:

- **Maximum number of elements:** This occurs when the binary heap is a perfect binary tree (all levels are completely filled). The number of nodes is the sum of 2^i for $i = 0$ to h , which evaluates to $2^{h+1} - 1$.
- **Minimum number of elements:** This occurs when the binary tree is complete, but the bottom level (h) has only the minimum required number of nodes, which is exactly 1. The number of nodes is the maximum of a heap of height $h - 1$ plus that 1 node: $(2^h - 1) + 1$, which equals 2^h .

(Note: If your course convention assumes a single node has a height of 1, the answers shift to 2^{h-1} for the minimum and $2^h - 1$ for the maximum).

(c) Stability of Insertion Sort and Quicksort

Insertion Sort is STABLE.

- **Justification:** Insertion sort iterates through the array and inserts each element into its proper place in the sorted portion. It strictly only shifts an element if it is *strictly greater* than the element being inserted. Equal elements do not cross over one another.
- **Example:** Consider the array `[4, 3a, 3b, 1]`.
 1. 4 is sorted.
 2. 3a is compared to 4. 4 shifts, 3a is inserted: `[3a, 4, 3b, 1]`.
 3. 3b is compared to 4 (shifts) and 3a (does not shift). 3b is inserted after 3a: `[3a, 3b, 4, 1]`.
 4. 1 is inserted at the front: `[1, 3a, 3b, 4]`. The relative order of 3a and 3b is maintained.

Quicksort is NOT STABLE.

- **Justification:** Standard, in-place quicksort partitions the array by swapping elements across the pivot. This long-distance swapping can easily invert the relative order of identical elements.
- **Example:** Consider the array `[3a, 3b, 1]`. Let's say 1 is chosen as the pivot (using the Lomuto partition scheme where the pivot is at the end).

1. The algorithm scans from left to right. Both 3a and 3b are greater than 1, so they stay in place during the scan.
2. At the end of the partition step, the pivot 1 is swapped with the first element greater than it to place the pivot in its correct sorted position.
3. 1 swaps with 3a, resulting in [1, 3b, 3a]. The relative order of the identical elements 3a and 3b has been reversed.

(d) Key Factors for Dynamic Programming

The two key properties that must be satisfied for an optimization problem to be solved efficiently using Dynamic Programming are:

1. **Optimal Substructure:** The optimal solution to the overall problem can be constructed from the optimal solutions of its subproblems.
2. **Overlapping Subproblems:** The problem can be broken down into subproblems which are reused several times, rather than generating new subproblems at every step (allowing the algorithm to memoize or tabulate answers).

(e) Uniqueness of Minimum Spanning Trees (MST)

Justification: The minimum spanning tree of a graph is unique *only* if all edge weights in the graph are distinct. If multiple edges share the same weight, an MST algorithm (like Kruskal's or Prim's) may have arbitrary choices when selecting an edge to add to the tree. Depending on which edge is chosen to break the tie, different configurations of spanning trees can emerge, all sharing the same minimum total weight.

Example with 5 nodes: Consider a graph structured as a simple ring (cycle) with 5 nodes (A, B, C, D, E) where every edge has a weight of 1.

- **Nodes:** {A, B, C, D, E}
- **Edges:** (A-B, 1), (B-C, 1), (C-D, 1), (D-E, 1), (E-A, 1)

To form a spanning tree, we must remove exactly one edge to break the cycle, leaving 4 edges. Because all edges weigh 1, the total weight of any spanning tree will be 4. We can create 5 different valid MSTs by dropping a different edge each time. Here are two examples:

- **MST 1:** Drop edge E-A. The tree contains edges {(A-B), (B-C), (C-D), (D-E)}.
- **MST 2:** Drop edge A-B. The tree contains edges {(B-C), (C-D), (D-E), (E-A)}.

You Asked 

(f) Would you use BFS to find the shortest path between two nodes in a weighted graph with arbitrary edge weights? Justify your answer with the help of a graph having at least 5 Vertices and at least 7 edges. (3)

(g) "Counting sort is a comparison sort algorithm." Yes or No. Justify your answer using the input $A = \langle 4, 3, 2, 4, 1, 5, 2, 4, 3 \rangle$ (3)

Here are the solutions to the continuation of your questions:

(f) BFS for Shortest Path in a Weighted Graph

Answer: No, you should not use Breadth-First Search (BFS) to find the shortest path in a graph with arbitrary edge weights.

Justification: BFS discovers the shortest path based strictly on the **number of edges** (or hops), assuming all edges have an equal weight of 1. It explores the graph level by level. In a weighted graph, a path with many edges might actually have a much smaller total weight than a path with very few edges. BFS will terminate early as soon as it finds the target node with the fewest edges, completely missing the true shortest path by weight.

Example Graph: Consider a graph with 5 vertices {A, B, C, D, E} and 7 edges, where we want to find the shortest path from **A** to **E**.

- **Edges and Weights:**

1. A $\rightarrow$ E (weight 100)
2. A $\rightarrow$ B (weight 1)
3. A $\rightarrow$ D (weight 10)
4. B $\rightarrow$ C (weight 1)
5. B $\rightarrow$ D (weight 2)
6. C $\rightarrow$ E (weight 1)
7. D $\rightarrow$ E (weight 10)

- **BFS Execution:**

- BFS starts at A.
- It explores A's direct neighbors: E, B, and D (all 1 edge away).
- BFS finds the target node **E** immediately at a distance of 1 edge. The path found is **A $\rightarrow$ E**, which has a massive total weight of **100**.

- **Actual Shortest Path:**

- If we calculate by total weight, the path **A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ E** requires 3 edges but has a total weight of only **3** (1 + 1 + 1).
- BFS fails to find this optimal path because it stops looking once it finds the fewer-hop path. (Algorithms like Dijkstra's or Bellman-Ford are required for weighted graphs).

(g) Is Counting Sort a Comparison Sort?

Answer: No.

Justification: A comparison sort (like QuickSort or MergeSort) determines the final sorted order by directly comparing elements against each other (e.g., asking "is $x < y$?"). Counting sort does not perform any pairwise comparisons. Instead, it uses the actual *values* of the elements as indexes (keys) into an auxiliary array to count their occurrences, mapping them directly to their final sorted positions.

Example using Input A = <4, 3, 2, 4, 1, 5, 2, 4, 3>:

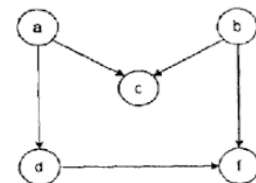
1. **Find the maximum:** The maximum value in the array is 5.

2. **Create a count array:** We create an auxiliary array `C` of size 6 (indexes 0 to 5), initialized to all zeros: `[0, 0, 0, 0, 0, 0]`.
3. **Count occurrences:** We iterate through `A` one by one. For each element x , we simply increment the value at index x in array `C`. We never compare two elements of `A` to each other.
 - Read 4: `C[4]` becomes 1
 - Read 3: `C[3]` becomes 1
 - Read 2: `C[2]` becomes 1
 - Read 4: `C[4]` becomes 2
 - ...and so on.
4. **Final Count Array:** After reading all of `A`, our count array `C` looks like this:
 - Index 0: 0
 - Index 1: 1 (Value 1 appears once)
 - Index 2: 2 (Value 2 appears twice)
 - Index 3: 2 (Value 3 appears twice)
 - Index 4: 3 (Value 4 appears three times)
 - Index 5: 1 (Value 5 appears once)

Because we sorted the array simply by tallying the frequency of each integer into a bucket array and then reading them out in order, we proved that no pairwise comparisons were necessary to establish the sorted order.

You Asked 

- (i) Find possible Topological sorts of the given directed acyclic graph. Give any four: (2)



Gemini 3.1 Pro Extended

Here is the step-by-step breakdown to find the topological sorts for the given Directed Acyclic Graph (DAG).

1. Analyze Dependencies

A topological sort is a linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u comes before v in the ordering. First, we identify the prerequisites (incoming edges) for each node:

- **Node a:** No dependencies (can be placed first).
- **Node b:** No dependencies (can be placed first).
- **Node c:** Must come after **a** and **b**.
- **Node d:** Must come after **a**.

- **Node f:** Must come after **b** and **d**.

2. Generate Valid Orderings

Because **a** and **b** have no incoming edges, any valid topological sort must start with either **a** or **b**.

Here are four possible topological sorts derived from these rules:

Option 1: a, b, c, d, f

- Start with **a** and **b**.
- With **a** and **b** placed, **c** and **d** are now free. We place **c**.
- Place **d**. Now **d** and **b** are placed, freeing **f**.
- Place **f**.

Option 2: a, b, d, c, f

- Start with **a** and **b**.
- Instead of **c**, we place **d** first (which is valid since **a** is already placed).
- Place **c** (valid since **a** and **b** are placed).
- Place **f** (valid since **b** and **d** are placed).

Option 3: b, a, d, f, c

- Start with **b** then **a**.
- Place **d** (valid since **a** is placed).
- Place **f** (valid since **b** and **d** are placed).
- Place **c** last (valid since both **a** and **b** were placed earlier).

Option 4: a, d, b, c, f

- Start with **a**.
- Place **d** immediately (valid since **a** is placed).
- Place **b** (no dependencies).
- Place **c** (valid since **a** and **b** are placed).
- Place **f** (valid since **b** and **d** are placed).

You Asked 

- (j) Give the worst case for the merge algorithm to merge two sorted arrays $A[1...k]$, $B[1...m]$ (where $n = k + m$) and give the total number of comparisons in the worst case (in terms of n). Merge the two sorted arrays $A = [3, 7, 9, 12, 14]$ and $B = [2, 5, 6, 10]$ using merge algorithm. How many number of comparisons are done by the algorithm in the above example? (6)

Part 1: Worst-Case Analysis of the Merge Algorithm

- **The Worst-Case Scenario:** The worst case for the merge algorithm occurs when the elements of the two sorted arrays **alternate** in their relative numerical order (e.g., $A = [1, 3, 5]$ and $B = [2, 4, 6]$). This configuration forces the algorithm to perform a comparison at almost every single step until one array runs out of elements, leaving exactly one element remaining in the other array.
- **Total Number of Comparisons:** If the sizes of the arrays are k and m , the maximum number of element-to-element comparisons required is $k + m - 1$. In terms of n (where $n = k + m$), the total number of comparisons in the worst case is: $n - 1$

Part 2: Step-by-Step Merging Example

Given the arrays:

- $A = [3, 7, 9, 12, 14]$ (size $k = 5$)
- $B = [2, 5, 6, 10]$ (size $m = 4$)
- Total elements $n = 5 + 4 = 9$

We track two pointers starting at the beginning of each array, comparing elements sequentially:

Step	Comparison	Action Taken	Resulting Merged Array	Remaining Elements in A & B
1	Compare $A[0] = 3$ and $B[0] = 2$	$2 < 3$, move 2	[2]	$A = [3, 7, 9, 12, 14]$, $B = [5, 6, 10]$
2	Compare $A[0] = 3$ and $B[1] = 5$	$3 < 5$, move 3	[2, 3]	$A = [7, 9, 12, 14]$, $B = [5, 6, 10]$
3	Compare $A[1] = 7$ and $B[1] = 5$	$5 < 7$, move 5	[2, 3, 5]	$A = [7, 9, 12, 14]$, $B = [6, 10]$
4	Compare $A[1] = 7$ and $B[2] = 6$	$6 < 7$, move 6	[2, 3, 5, 6]	$A = [7, 9, 12, 14]$, $B = [10]$
5	Compare $A[1] = 7$ and $B[3] = 10$	$7 < 10$, move 7	[2, 3, 5, 6, 7]	$A = [9, 12, 14]$, $B = [10]$
6	Compare $A[2] = 9$ and $B[3] = 10$	$9 < 10$, move 9	[2, 3, 5, 6, 7, 9]	$A = [12, 14]$, $B = [10]$
7	Compare $A[3] = 12$ and $B[3] = 10$	$10 < 12$, move 10	[2, 3, 5, 6, 7, 9, 10]	$A = [12, 14]$, $B = []$ (Exhausted)

Note on Remaining Elements: At Step 7, Array B is completely exhausted. The remaining unpicked elements of Array A ($[12, 14]$) are simply appended directly to the end of the merged list. **No further comparisons are required** for these trailing elements.

- **Final Sorted Array:** [2, 3, 5, 6, 7, 9, 10, 12, 14]

- **Total Number of Comparisons Done: 7 comparisons**

You Asked 

(k) Consider the Interval Scheduling problem wherein we are given a resource and a set of requests each having a start time and a finish time. The goal is to maximize the number of requests scheduled. Show that the following greedy strategy does not give an optimal solution for the above problem.

Greedy strategy: Select the request with fewest number of incompatible requests. (3)

Gemini 3.5 Flash Extended

To show that the greedy strategy "select the request with the fewest number of incompatible requests" does not always yield an optimal solution, we can construct a counterexample where picking the interval with the minimum conflicts eliminates a larger set of mutually compatible intervals.

The Counterexample

Imagine a set of intervals distributed across a timeline. We want to design it such that a single "bait" interval G has the fewest conflicts, but choosing it destroys the chance to pick a larger, optimal chain of intervals.

1. Definition of Intervals

Let's define a set of requests with their [start_time, end_time] :

- **Optimal Set (4 independent intervals):**
 - $O_1 = [1, 3]$
 - $O_2 = [4, 6]$
 - $O_3 = [7, 9]$
 - $O_4 = [10, 12]$
- **Greedy Bait Interval:**
 - $G = [5, 8]$ (Note: This overlaps with O_2 and O_3)
- **Filler Intervals (added strictly to increase conflict counts of other nodes):**
 - Three intervals overlapping only O_1 : $F_1, F_2, F_3 = [1.5, 2.5]$
 - Three intervals overlapping only O_2 : $F_4, F_5, F_6 = [4.1, 4.2]$
 - Three intervals overlapping only O_3 : $F_7, F_8, F_9 = [8.1, 8.2]$
 - Three intervals overlapping only O_4 : $F_{10}, F_{11}, F_{12} = [10.5, 11.5]$

2. Count of Incompatible Requests (Conflicts)

Let's calculate the number of conflicts for each interval type to see what the greedy strategy chooses:

Interval	Incompatible With...	Total Conflict Count
G	O_2, O_3	2 (Fewest conflicts!)
O_1	F_1, F_2, F_3	3
F_1 to F_3	O_1 and the other 2 fillers in its cluster	3
O_2	G, F_4, F_5, F_6	4
F_4 to F_6	O_2 and the other 2 fillers in its cluster	3
O_3	G, F_7, F_8, F_9	4
F_7 to F_9	O_3 and the other 2 fillers in its cluster	3
O_4	F_{10}, F_{11}, F_{12}	3
F_{10} to F_{12}	O_4 and the other 2 fillers in its cluster	3

3. Comparison of Outcomes

- **Greedy Strategy Execution:** 1. The algorithm selects G first because it has the absolute minimum number of conflicts (2). 2. Selecting G immediately eliminates its incompatible neighbors: O_2 and O_3 . 3. From the remaining valid timeline blocks (the O_1 cluster and the O_4 cluster), the algorithm can pick at most 2 more non-overlapping intervals (e.g., O_1 and O_4). 4. **Total scheduled by Greedy = 3 requests** (G, O_1, O_4).
- **Optimal Solution:**
 - We can completely ignore G and all the filler clusters, choosing the entirely compatible chain: $\{O_1, O_2, O_3, O_4\}$.
 - **Total scheduled by Optimal Solution = 4 requests.**

Conclusion

Since the Greedy strategy schedules 3 requests while the optimal layout permits 4, the strategy of choosing the request with the fewest incompatible requests **is not optimal**.

You Asked 

3. (a) Illustrate the operation of BUILD-MAX-HEAP on the array $A = \langle 4, 3, 17, 10, 28, 19, 6, 12, 7 \rangle$. Write down the total number of comparisons done by BUILD-MAX-HEAP. (5)

Here is the step-by-step illustration of the BUILD-MAX-HEAP operation and the calculation for the total number of comparisons.

This solution uses standard 1-based indexing for the array, where the left child of node i is at $2i$ and the right child is at $2i + 1$.

Initial Setup

- **Initial Array:** $A = [4, 3, 17, 10, 28, 19, 6, 12, 7]$
 - **Array Length (n):** 9
 - The BUILD-MAX-HEAP algorithm works bottom-up, calling MAX-HEAPIFY on all non-leaf nodes. It starts at index $i = \lfloor n/2 \rfloor = 4$ and goes down to 1.
-

Step-by-Step Operation

Step 1: $i = 4$ (Node value = 10)

- **Subtree:** Parent is 10 ($A[4]$). Left child is 12 ($A[8]$). Right child is 7 ($A[9]$).
- **Comparisons:**
 1. Compare left child with parent: $12 > 10$. Largest is left child.
 2. Compare right child with largest: $7 \not> 12$. Largest remains left child.
- **Action:** Swap $A[4]$ and $A[8]$.
- **Array becomes:** $[4, 3, 17, 12, 28, 19, 6, 10, 7]$
- **Comparisons made:** 2

Step 2: $i = 3$ (Node value = 17)

- **Subtree:** Parent is 17 ($A[3]$). Left child is 19 ($A[6]$). Right child is 6 ($A[7]$).
- **Comparisons:**
 1. Compare left child with parent: $19 > 17$. Largest is left child.
 2. Compare right child with largest: $6 \not> 19$. Largest remains left child.
- **Action:** Swap $A[3]$ and $A[6]$.
- **Array becomes:** $[4, 3, 19, 12, 28, 17, 6, 10, 7]$
- **Comparisons made:** 2

Step 3: $i = 2$ (Node value = 3)

- **Subtree:** Parent is 3 ($A[2]$). Left child is 12 ($A[4]$). Right child is 28 ($A[5]$).
- **Comparisons:**
 1. Compare left child with parent: $12 > 3$. Largest is left child.
 2. Compare right child with largest: $28 > 12$. Largest is right child.
- **Action:** Swap $A[2]$ and $A[5]$.
- **Array becomes:** $[4, 28, 19, 12, 3, 17, 6, 10, 7]$
- **Comparisons made:** 2 (The recursive call to MAX-HEAPIFY(5) does nothing because node 5 has no children, so 0 comparisons are added there).

Step 4: $i = 1$ (Node value = 4)

This step cascades downward because the swapped value disrupts the lower heap structure.

- **Initial Subtree:** Parent is 4 ($A[1]$) . Left child is 28 ($A[2]$) . Right child is 19 ($A[3]$) .
- **Comparisons (Level 1):**
 1. $28 > 4$ (Largest is left child).
 2. $19 \not> 28$ (Largest remains left child).
- **Action:** Swap $A[1]$ and $A[2]$.
- **Array becomes:** $[28, 4, 19, 12, 3, 17, 6, 10, 7]$
- **Recursive Call at $i = 2$ (Node value = 4):** Left child is 12 ($A[4]$) . Right child is 3 ($A[5]$) .
- **Comparisons (Level 2):** 3. $12 > 4$ (Largest is left child). 4. $3 \not> 12$ (Largest remains left child).
- **Action:** Swap $A[2]$ and $A[4]$.
- **Array becomes:** $[28, 12, 19, 4, 3, 17, 6, 10, 7]$
- **Recursive Call at $i = 4$ (Node value = 4):** Left child is 10 ($A[8]$) . Right child is 7 ($A[9]$) .
- **Comparisons (Level 3):** 5. $10 > 4$ (Largest is left child). 6. $7 \not> 10$ (Largest remains left child).
- **Action:** Swap $A[4]$ and $A[8]$.
- **Array becomes:** $[28, 12, 19, 10, 3, 17, 6, 4, 7]$
- **Comparisons made:** 6 (2 at each of the 3 levels)

Final Output

Final Max-Heap Array: $A = [28, 12, 19, 10, 3, 17, 6, 4, 7]$

Total Number of Comparisons: To find the total, we sum the comparisons from every execution step of MAX-HEAPIFY :

- At $i = 4$: 2 comparisons
- At $i = 3$: 2 comparisons
- At $i = 2$: 2 comparisons
- At $i = 1$: 6 comparisons
- **Total:** $2 + 2 + 2 + 6 = 12$ element comparisons

You Asked 

5. (a) A Shopkeeper has n empty boxes and M number of balls. Let $\{K_1, K_2, \dots, K_n\}$ denote the number of balls that each box can store.

Given M and $\{K_1, K_2, \dots, K_n\}$, describe a greedy algorithm which determines the minimum number of boxes needed to store the balls. Give time complexity of the algorithm. (4)

Here is the description and analysis of the greedy algorithm for the given problem.

Greedy Strategy

To minimize the total number of boxes used, the optimal greedy choice is to always select the box with the **largest remaining capacity**. By prioritizing the largest boxes first, we pack the maximum possible number of balls at each step, thereby exhausting the M balls using the fewest number of containers.

Algorithm Description

1. **Sort the Capacities:** Sort the array of box capacities $\{K_1, K_2, \dots, K_n\}$ in **descending order**. Let's call this sorted array K_{sorted} .
2. **Initialize Variables:** * Set a counter for boxes used: `boxes_used = 0` .
 - Keep track of the remaining balls: `remaining_balls = M` .
3. **Iterate and Fill:** Loop through each capacity K_i in the sorted array K_{sorted} :
 - If `remaining_balls <= 0` , stop the loop. All balls have been successfully stored.
 - Otherwise, place as many balls as possible into the current box. Subtract K_i from `remaining_balls` (i.e., `remaining_balls = remaining_balls - K_i`).
 - Increment `boxes_used` by 1.
4. **Final Check:** * After the loop finishes, if `remaining_balls > 0` , it means the total capacity of all boxes combined is strictly less than M . You cannot store all the balls.
 - Otherwise, output the value of `boxes_used` .

Time Complexity Analysis

- **Sorting Step:** Sorting the array of n elements takes $O(n \log n)$ time using an efficient comparison-based sorting algorithm like Merge Sort or Heap Sort.
- **Iteration Step:** Iterating through the sorted array to subtract capacities takes at most $O(n)$ time in the worst-case scenario (where you have to use almost all or all of the boxes).

Combining these steps, the dominant operation is the sorting phase. Therefore, the overall **Time Complexity** of this greedy algorithm is: $O(n \log n)$

You Asked 

(b) Design a $O(|V|+|E|)$ time algorithm to find whether a given undirected graph is bipartite (where V is the set of vertices, E is the set of edges of the graph). (4)

Here is the algorithm to determine if an undirected graph is bipartite in $O(|V| + |E|)$ time.

Core Concept: 2-Coloring

A graph is bipartite if and only if its vertices can be divided into two disjoint sets such that no two adjacent vertices share the same set. This is equivalent to saying the graph is **2-colorable**. We can use a Breadth-First Search (BFS) traversal to attempt to color the graph using two alternating colors (e.g., 0 and 1).

The Algorithm (BFS Approach)

1. Initialization:

- Create a `color` array of size $|V|$ and initialize all entries to `-1` (indicating uncolored).
- Create a standard FIFO queue for the BFS.

2. Handle Disconnected Components:

- Because the graph might not be fully connected, iterate through every vertex $v \in V$.
- If `color[v]` is `-1` (uncolored), start a BFS traversal from v .

3. Breadth-First Search (BFS) Traversal:

- Assign a starting color to v : `color[v] = 0`.
- Enqueue v .
- While the queue is not empty:
 - Dequeue a vertex, let's call it u .
 - Iterate through all adjacent neighbors of u . For each neighbor w :
 - **Case A (Uncolored):** If `color[w] == -1`, assign it the opposite color of u (i.e., `color[w] = 1 - color[u]`). Enqueue w .
 - **Case B (Colored with the same color):** If `color[w] == color[u]`, an odd cycle has been found. The graph cannot be bipartite. **Return False**.
 - **Case C (Colored with opposite color):** If `color[w] != color[u]`, do nothing and continue.

4. Completion:

- If all components are successfully traversed and colored without any conflicts (i.e., no two adjacent vertices share the same color), **Return True**.

Complexity Analysis

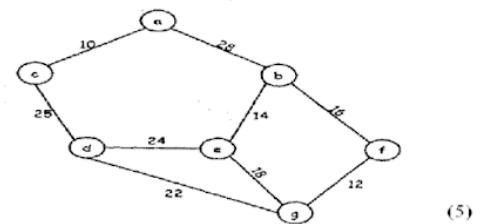
- **Time Complexity:** $O(|V| + |E|)$
 - In the worst-case scenario, every vertex is pushed to and popped from the queue exactly once, which takes $O(|V|)$ time.
 - For every dequeued vertex, we iterate through its adjacency list. In an undirected graph, each edge is checked exactly twice (once from each endpoint). This takes $O(|E|)$ time.
 - Summing these together gives the required linear time bound of $O(|V| + |E|)$.
- **Space Complexity:** $O(|V|)$

- The `color` array requires $\mathcal{O}(|V|)$ space.
- The BFS queue will store at most all the vertices in the graph simultaneously, requiring an additional $\mathcal{O}(|V|)$ space.

You Asked 

(c) Give space requirements of adjacency matrix and adjacency list representation having m edges and n vertices. (2)

6. (a) "Prim's algorithm only include an edge in the Minimum Spanning tree when it is justified by the Cut property." State the Cut Property. Justify the above statement on the given graph showing cuts in all the intermediate steps.



 Gemini 3.1 Pro

Here are the solutions to the questions presented in the image:

(c) Space Requirements for Graph Representations

For a graph with n vertices and m edges:

- **Adjacency Matrix:** The space requirement is $\mathcal{O}(n^2)$.
 - *Justification:* It uses an $n \times n$ matrix (a 2D array) where rows and columns represent vertices, and the cells store edge weights or boolean values indicating the presence of an edge. The space is dependent entirely on the number of vertices, regardless of how few edges there might be.
- **Adjacency List:** The space requirement is $\mathcal{O}(n + m)$.
 - *Justification:* It uses an array (or list) of size n to represent the vertices. Each entry points to a linked list containing its neighbors. In a directed graph, the total number of nodes across all lists is m . In an undirected graph, it is $2m$. Therefore, the space scales linearly with both vertices and edges, making it highly efficient for sparse graphs.

(6a) Prim's Algorithm and the Cut Property

The Cut Property: A "cut" partitions the vertices of a graph into two disjoint sets, say S and $V - S$. An edge is said to "cross the cut" if one of its endpoints is in S and the other is in $V - S$. The **Cut Property** states that for any cut of the graph, the crossing edge with the absolute minimum weight is guaranteed to be part of the Minimum Spanning Tree (MST).

Justification in Prim's Algorithm: Prim's algorithm builds the MST by directly applying this property at every step. It maintains a set of visited vertices (S , the growing tree) and unvisited vertices ($V - S$). The boundary between these two sets forms a

cut. In each iteration, Prim's examines all edges crossing this cut and selects the one with the lowest weight to add to the MST, moving its connecting vertex into S .

Step-by-Step Justification on the Given Graph: Let's trace the algorithm, starting arbitrarily at vertex **a**.

- **Initialization:** $S = \{a\}$
 - $V - S = \{b, c, d, e, f, g\}$
- **Step 1:**
 - **The Cut:** $S = \{a\}$ vs $V - S = \{b, c, d, e, f, g\}$
 - **Crossing Edges:** $(a, c) = 10$, $(a, b) = 28$
 - **Min Edge Selected:** **(a, c)** with weight 10.
 - **Update S:** $S = \{a, c\}$
- **Step 2:**
 - **The Cut:** $S = \{a, c\}$ vs $V - S = \{b, d, e, f, g\}$
 - **Crossing Edges:** $(a, b) = 28$, $(c, d) = 25$
 - **Min Edge Selected:** **(c, d)** with weight 25.
 - **Update S:** $S = \{a, c, d\}$
- **Step 3:**
 - **The Cut:** $S = \{a, c, d\}$ vs $V - S = \{b, e, f, g\}$
 - **Crossing Edges:** $(a, b) = 28$, $(d, e) = 24$, $(d, g) = 22$
 - **Min Edge Selected:** **(d, g)** with weight 22.
 - **Update S:** $S = \{a, c, d, g\}$
- **Step 4:**
 - **The Cut:** $S = \{a, c, d, g\}$ vs $V - S = \{b, e, f\}$
 - **Crossing Edges:** $(a, b) = 28$, $(d, e) = 24$, $(g, e) = 18$, $(g, f) = 12$
 - **Min Edge Selected:** **(g, f)** with weight 12.
 - **Update S:** $S = \{a, c, d, g, f\}$
- **Step 5:**
 - **The Cut:** $S = \{a, c, d, g, f\}$ vs $V - S = \{b, e\}$
 - **Crossing Edges:** $(a, b) = 28$, $(d, e) = 24$, $(g, e) = 18$, $(f, b) = 16$
 - **Min Edge Selected:** **(f, b)** with weight 16.
 - **Update S:** $S = \{a, c, d, g, f, b\}$
- **Step 6:**
 - **The Cut:** $S = \{a, c, d, g, f, b\}$ vs $V - S = \{e\}$
 - **Crossing Edges:** $(d, e) = 24$, $(g, e) = 18$, $(b, e) = 14$
 - *(Note: (a,b) no longer crosses the cut as both are in S)*
 - **Min Edge Selected:** **(b, e)** with weight 14.
 - **Update S:** $S = \{a, c, d, g, f, b, e\}$

All vertices are now in set S . The algorithm terminates, having constructed the Minimum Spanning Tree using edges: $\{(a, c), (c, d), (d, g), (g, f), (f, b), (b, e)\}$ with a total weight of 99.

You Asked 

(c) Let G be a graph with n vertices and m edges. What is the upper bound on the running time of Depth First Search on G , where G is represented as an adjacency matrix? (2)

 Gemini 3.1 Pro

The upper bound on the running time of Depth First Search (DFS) on a graph represented as an adjacency matrix is $O(n^2)$.

Justification:

- **Matrix Structure:** An adjacency matrix for a graph with n vertices is an $n \times n$ grid.
- **Vertex Discovery:** During a DFS traversal, the algorithm visits every vertex in the graph.
- **Finding Neighbors:** To discover which nodes are connected to the current vertex being visited, the algorithm must scan the entirety of that vertex's row in the matrix. Because the row has length n , this scan takes $O(n)$ time.
- **Total Time:** Since the algorithm performs an $O(n)$ row scan for each of the n vertices, the total time taken to explore all edges is $n \times O(n) = O(n^2)$.

Unlike an adjacency list representation (which runs in $O(n + m)$ time because it only iterates over actual existing edges), an adjacency matrix forces the algorithm to check every possible edge combination (including non-existent edges) to determine the graph's structure, completely dominating the time complexity regardless of the actual number of edges m .

You Asked 

Capacity of the boat = 4kg

Items	Weights
Item1	3 kg
Item2	2 kg
Item3	1 kg

7. (a) A boat has a capacity C to carry load. There are n number of items each having certain weight W_i associated with it. The goal is to select the set of items that the boat should carry so that it could carry maximum load i.e. sum of the weights of selected items should be maximum without its capacity C .
Design a Polynomial time Dynamic Programming algorithm for the problem. Derive the time complexity of the above algorithm.
Run this algorithm on the sample instance given below to find the optimal solution. (10)

 Gemini 3.1 Pro

Dynamic Programming Algorithm Design

This problem is a variation of the classic **0/1 Knapsack Problem**, known as the **Subset Sum Problem**. Since the goal is to maximize the total weight carried without exceeding the capacity C , we can treat the weight of each item as its "value".

Let $DP[i][j]$ represent the maximum load the boat can carry using a subset of the first i items, such that the total weight does not exceed the current capacity j .

1. State Definition:

- i represents the items considered so far ($1 \leq i \leq n$).
- j represents the current capacity being evaluated ($0 \leq j \leq C$).

2. Initialization (Base Cases):

- $DP[0][j] = 0$ for all $0 \leq j \leq C$ (With 0 items, the load is 0).
- $DP[i][0] = 0$ for all $0 \leq i \leq n$ (With 0 capacity, the load is 0).

3. Recurrence Relation: For each item i with weight W_i :

- Case 1 (Item is too heavy):** If $W_i > j$, the item cannot be included. The maximum load remains the same as it was for the previous $i - 1$ items at this capacity. $DP[i][j] = DP[i - 1][j]$
- Case 2 (Item fits):** If $W_i \leq j$, we must choose the maximum between two options:
 - Excluding the item: $DP[i - 1][j]$
 - Including the item: The weight of the item (W_i) plus the maximum weight achievable with the remaining capacity ($j - W_i$) using the previous items. $DP[i][j] = \max(DP[i - 1][j], DP[i - 1][j - W_i] + W_i)$

Time Complexity Analysis

The algorithm builds a 2D table `DP` with dimensions $(n + 1) \times (C + 1)$.

- There are a total of $(n + 1) \times (C + 1)$ cells to compute.
- Computing the value for each cell takes constant time, $O(1)$, because it only requires a simple comparison (`max` function) of previously computed values.

Therefore, the time complexity of this algorithm is $O(nC)$. This is known as a pseudo-polynomial time complexity because it depends not just on the number of items (n), but also on the numeric magnitude of the capacity (C).

Execution on the Sample Instance

Given Data:

- Capacity $C = 4$ kg
- Number of items $n = 3$
- Weights: $W_1 = 3, W_2 = 2, W_3 = 1$

We initialize a DP table of size 4×5 (from $i = 0$ to 3 and $j = 0$ to 4).

i (Items) / j (Capacity)	0	1	2	3	4
0 (No items)	0	0	0	0	0
1 ($W_1 = 3$)	0	0	0	3	3
2 ($W_2 = 2$)	0	0	2	3	3
3 ($W_3 = 1$)	0	1	2	3	4

Step-by-Step Table Generation:

- **Row 1** ($W_1 = 3$): Fits only when capacity $j \geq 3$.
 - $j = 3$: $\max(0, DP[0][0] + 3) = 3$
 - $j = 4$: $\max(0, DP[0][1] + 3) = 3$
- **Row 2** ($W_2 = 2$): * $j = 2$: $\max(DP[1][2], DP[1][0] + 2) = \max(0, 2) = 2$
 - $j = 3$: $\max(DP[1][3], DP[1][1] + 2) = \max(3, 2) = 3$
 - $j = 4$: $\max(DP[1][4], DP[1][2] + 2) = \max(3, 2) = 3$
- **Row 3** ($W_3 = 1$):
 - $j = 1$: $\max(DP[2][1], DP[2][0] + 1) = \max(0, 1) = 1$
 - $j = 2$: $\max(DP[2][2], DP[2][1] + 1) = \max(2, 1) = 2$
 - $j = 3$: $\max(DP[2][3], DP[2][2] + 1) = \max(3, 3) = 3$
 - $j = 4$: $\max(DP[2][4], DP[2][3] + 1) = \max(3, 3 + 1) = 4$

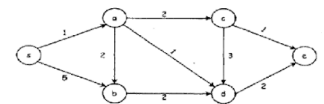
Finding the Optimal Items (Traceback): To find which items are included, we start from the bottom-right cell $DP[n][C]$ and trace backwards.

1. Start at $DP[3][4] = 4$.
2. Does $DP[3][4] == DP[2][4]$? ($4 == 3$ is False). This means **Item 3 is included**.
 - Subtract its weight from capacity: $4 - 1 = 3$. Move to $DP[2][3]$.
3. Does $DP[2][3] == DP[1][3]$? ($3 == 3$ is True). This means **Item 2 is NOT included**.
 - Capacity remains 3. Move to $DP[1][3]$.
4. Does $DP[1][3] == DP[0][3]$? ($3 == 0$ is False). This means **Item 1 is included**.

Final Answer: The optimal solution is to select **Item 1** (3 kg) and **Item 3** (1 kg). The maximum load the boat can carry is **4 kg**, which perfectly fills its capacity.

You Asked 

8. (a) The following graph represents network of airports that are connected to each other. Each edge represents the distance (in multiples of 1000 miles) covered by the flight to travel from one airport to another and vertices represents the airports. A flight starts from airport s and has to reach destinations c , d and e . Run an efficient algorithm to find the route taken by the flight to each of its destinations in minimum possible time. Show all the intermediate steps taken by the algorithm. Also derive its time complexity. (15)



Gemini 3.1 Pro

To find the routes with the minimum possible time (shortest path) from a single source node (s) to multiple destinations in a weighted directed graph with non-negative edge weights, the most efficient standard algorithm is **Dijkstra's Algorithm**.

Execution of Dijkstra's Algorithm (Intermediate Steps)

We will track the shortest known distance to each node from s and update these distances as we explore the graph.

Initialization: * Set the distance to the source node s to 0.

- Set the distance to all other nodes (a, b, c, d, e) to ∞ .

- Keep a record of the "predecessor" node to reconstruct the path later.

Step	Current Node	s	a	b	c	d	e	Unvisited Nodes
0	(Start)	0	∞	∞	∞	∞	∞	{s, a, b, c, d, e}
1	Explore s	0	1 (from s)	5 (from s)	∞	∞	∞	{a, b, c, d, e}
2	Explore a	0	1	3 (from a)	3 (from a)	2 (from a)	∞	{b, c, d, e}
3	Explore d	0	1	3	3	2	4 (from d)	{b, c, e}
4	Explore b	0	1	3	3	2	4	{c, e}
5	Explore c	0	1	3	3	2	4	{e}
6	Explore e	0	1	3	3	2	4	$\emptyset$

Note on Step 5: When exploring c, the path to e via c ($3 + 1 = 4$) ties with the existing path via d ($2 + 2 = 4$). Both are valid optimal paths.

Final Routes and Minimum Distances

Based on the predecessor tracking in the steps above, here are the optimal routes and total distances (in multiples of 1000 miles) to destinations c, d, and e:

- Destination c:** * Route: $s \rightarrow a \rightarrow c$
 - Minimum Time/Distance: 3
- Destination d:**
 - Route: $s \rightarrow a \rightarrow d$
 - Minimum Time/Distance: 2
- Destination e:**
 - Route: $s \rightarrow a \rightarrow d \rightarrow e$ OR $s \rightarrow a \rightarrow c \rightarrow e$
 - Minimum Time/Distance: 4

Time Complexity Derivation

The time complexity of Dijkstra's Algorithm depends entirely on the data structures used to represent the graph and to extract the minimum distance node (the priority queue). Let $|V|$ be the number of vertices and $|E|$ be the number of edges.

- Extracting the minimum distance:** The algorithm extracts the node with the minimum distance from the unvisited set exactly $|V|$ times.
- Edge relaxation:** For every extracted node, the algorithm iterates through its outgoing edges. Across the entire execution, this means every edge in the graph is checked exactly once, resulting in $|E|$ operations.

Complexity based on Data Structure:

- **Adjacency Matrix + Linear Array (Basic):** Finding the minimum node takes $O(|V|)$ time, done $|V|$ times. Checking edges takes $O(|V|)$ per node. Total time: $O(|V|^2)$.
- **Adjacency List + Binary Min-Heap (Standard Efficient):** Finding and extracting the minimum takes $O(\log |V|)$ time, done $|V|$ times. Updating the distance (decrease-key operation) for each edge takes $O(\log |V|)$ time, done up to $|E|$ times. Total time: $O((|V| + |E|) \log |V|)$.
- **Adjacency List + Fibonacci Heap (Most Efficient Theoretical):** Decrease-key operations take $O(1)$ amortized time. Total time is reduced to $O(|E| + |V| \log |V|)$.

You Asked 

(b) There are n Jobs where each job starts at time s_i and finishes at time f_i . There is a profit associated with each job. The goal is to find a subset of non-overlapping jobs such that the sum of their profits is maximum.

Given below is the instance of the problem :

Job Number	Start Time (s_i)	Finish time (f_i)	Profit (p_i)
Job1	0	6	60
Job2	1	4	30
Job3	3	5	10
Job4	5	7	30
Job5	5	9	50
Job6	7	8	10

Give a Dynamic programming iterative solution for the above problem. Explain the recurrence relation used in the solution.

Show that the "Optimal solution to the above problem contains within it optimal solution to its subproblems." With reference to the above example. (5)

 Gemini 3.1 Pro

Here is the step-by-step Dynamic Programming (DP) solution for the Weighted Job Scheduling problem, followed by the explanation of the recurrence relation and optimal substructure.

1. Preparation: Sorting and Finding Compatibilities

To apply Dynamic Programming iteratively, we must first sort the jobs by their **Finish Time (f_i)** in non-decreasing order. This ensures we process smaller time intervals before larger ones.

Let's define the sorted jobs as $j_1, j_2, \dots, j_6$:

Sorted Index (i)	Original Job	Start (s_i)	Finish (f_i)	Profit (p_i)
1	Job 2	1	4	30
2	Job 3	3	5	10
3	Job 1	0	6	60

Sorted Index (i)	Original Job	Start (s_i)	Finish (f_i)	Profit (p_i)
4	Job 4	5	7	30
5	Job 6	7	8	10
6	Job 5	5	9	50

Next, for each job i , we find $p(i)$, which represents the index of the latest job (prior to i) that is compatible with job i (meaning it finishes before or exactly when job i starts).

- $p(1) = 0$ (No previous jobs)
- $p(2) = 0$ (Job 1 finishes at 4, Job 2 starts at 3. Overlap.)
- $p(3) = 0$ (Starts at 0, overlaps with 1 and 2.)
- $p(4) = 2$ (Job 4 starts at 5. Job 2 finishes at 5. They are compatible.)
- $p(5) = 4$ (Job 6 starts at 7. Job 4 finishes at 7. Compatible.)
- $p(6) = 2$ (Job 5 starts at 5. Job 2 finishes at 5. Compatible.)

2. The Recurrence Relation

Let $OPT(i)$ represent the maximum profit achievable using a non-overlapping subset of the first i jobs. For any job i , we have two choices:

1. **Include Job i :** We gain its profit p_i , plus the optimal profit from the remaining compatible jobs before it, which is $OPT(p(i))$.
2. **Exclude Job i :** We don't take it, meaning the maximum profit is just the optimal profit achieved by the previous $i - 1$ jobs, which is $OPT(i - 1)$.

The optimal choice is simply the maximum of these two options. The recurrence relation is:

$$OPT(i) = \max(p_i + OPT(p(i)), OPT(i - 1))$$

Base Case: $OPT(0) = 0$ (0 profit with 0 jobs).

3. Iterative DP Solution

Using the recurrence relation and our calculated $p(i)$ values, we build the DP table bottom-up:

- $OPT(0) = 0$
- $OPT(1) = \max(30 + OPT(0), OPT(0)) = \max(30 + 0, 0) = 30$
- $OPT(2) = \max(10 + OPT(0), OPT(1)) = \max(10 + 0, 30) = 30$
- $OPT(3) = \max(60 + OPT(0), OPT(2)) = \max(60 + 0, 30) = 60$
- $OPT(4) = \max(30 + OPT(2), OPT(3)) = \max(30 + 30, 60) = \max(60, 60) = 60$
- $OPT(5) = \max(10 + OPT(4), OPT(4)) = \max(10 + 60, 60) = \max(70, 60) = 70$
- $OPT(6) = \max(50 + OPT(2), OPT(5)) = \max(50 + 30, 70) = \max(80, 70) = 80$

The maximum profit possible is **80**. (Tracing backwards: 80 comes from including Job 6 and Job 2, which corresponds to Original Job 5 and Original Job 2. Their profits are $50 + 30 = 80$, and their times [1-4] and [5-9] do not overlap).

4. Proof of Optimal Substructure

Optimal Substructure means that an optimal solution to the overall problem can be constructed directly from the optimal solutions to its smaller subproblems. We prove this via a "cut-and-paste" contradiction argument using our specific example.

Let's look at the optimal solution for all 6 jobs, which yielded $OPT(6) = 80$. Based on our recurrence, this maximum was achieved by choosing to *include* job 6, meaning the total profit was formed by $p_6 + OPT(p(6))$, which equates to $50 + OPT(2)$.

For optimal substructure to hold, $OPT(2)$ **must** be the absolute maximum profit possible from the first 2 jobs. We know $OPT(2) = 30$.

Proof by Contradiction: Suppose that $OPT(2) = 30$ is *not* the optimal solution for the subproblem of the first 2 jobs. Suppose there existed some hypothetical better arrangement of the first 2 jobs that yielded a profit of 40 instead of 30.

If this were true, we could "cut" out our old sub-solution (30) and "paste" in this new, better sub-solution (40) into our main equation for job 6. The new total profit would become: $p_6 + (\text{New Sub-Solution}) = 50 + 40 = 90$.

However, this is a contradiction! We already mathematically established through exhaustive comparison that the absolute maximum possible profit for all 6 jobs is 80. You cannot exceed 80. Therefore, our assumption that a better sub-solution exists must be false.

Because we cannot improve the overall optimal solution by replacing a subproblem's solution, the optimal solution to the overall problem **must** contain within it the optimal solutions to its subproblems.

json

```
{"component":"LlmGeneratedComponent","props":{"height":"800px","prompt":"Create an interactive visualizer for the Weighted Interval Scheduling dynamic programming algorithm. Objective: Allow users to step through the DP array calculation to understand optimal substructure. Data State: Initialize with 6 jobs: {id: 1, start:1, end:4, profit:30}, {id: 2, start:3, end:5, profit:10}, {id: 3, start:0, end:6, profit:60}, {id: 4, start:5, end:7, profit:30}, {id: 5, start:7, end:8, profit:10}, {id: 6, start:5, end:9, profit:50}. These are already sorted by end time. Strategy: Standard Layout (Sims). Inputs: A 'Next Step' and 'Previous Step' button to iterate through i = 1 to 6. Behavior: Display a visual timeline/Gantt chart of the jobs at the top. Below, display the DP array calculation step-by-step. For the current step 'i', visually highlight the current job 'i' on the chart, highlight its latest non-conflicting job p(i) if it exists, and explicitly spell out the calculation: OPT(i) = max(Profit(i) + OPT(p(i)), OPT(i-1)). When the sequence reaches the end, highlight the final optimal subset of jobs on the chart.", "id":"im_644a1a4757add4db"}}
```